SPORTScore | R&D Team
Scientific Research Report

TinyML for UAV-Based Power-Line Inspection:
An Architecture-Aware Prune–Distill–Quantize Framework for Insulator Detection and Condition Assessment



Prepared by: Meryem Labyad
Supervisor: Noureddine Mohtaram
Research team: SPORTScore R&D
Research period: 2026
Institution: EMG Rabat – Cycle Ingénieur, Génie Informatique

September 2026

Abstract

Power-line inspection increasingly relies on images taken by UAVs, but the detectors that read these images are large and difficult to run on the small computers a drone can carry. This work studies how far a Faster R-CNN insulator detector can be compressed with four ordered steps: structured pruning, recalibration of the normalization statistics, distillation from the original network, and INT8 quantization. Channels are ranked with a gradient-based (Taylor) score, the dependencies between layers are handled when channels are removed, and the stem, the region proposal network, and the RoI head can be protected. The question behind the study is whether the usefulness of compression depends on where capacity is removed and not only on how much.

All results are on a held-out set of 320 images from the IDID V1.2 dataset, with a single seed unless stated. The moderate configuration removes 41.8% of the parameters and 30% of the GFLOPs with no loss of $AP_{50:95}$ (retention 101.1%; 95.6% when compared with the teacher fine-tuned with the same recipe, which itself gains 4.1 AP points). The aggressive configuration (57.6% fewer parameters) keeps 85.4% and the maximal one (61.7%) keeps 80.8%. Quantizing the stem is costly (about 20 retention points), so it stays in floating point; the final INT8 model keeps 80.2% of the teacher AP and is 18.2 MB, 9.1 times smaller than the teacher. Distillation, more than recalibration, explains the recovery after pruning. At a matched 57.6% reduction, uniform allocation (98.8%) beat both the protected allocation (85.4%) and the head-protected one (82.9%), so protecting the prediction modules did not help here. The loss from compression is concentrated on the damage classes and comes mostly from confusion between classes. A second study applies the same framework to ResNet-18 on Imagenette (81% fewer parameters, 92.3% accuracy after INT8 against 95.0%, 20.5 times smaller). Latency was measured on laptop and cloud CPUs only; no embedded hardware was used.

Keywords: TinyML; Edge AI; UAV inspection; power-line inspection; insulator detection; condition assessment; structured pruning; knowledge distillation; INT8 quantization; Faster R-CNN.

Contents

Abstract **1**

1 Introduction **6**

1.1 Criticality of Power-Line Inspection . . . 6
1.2 Why UAV-Based Inspection? . . . 6
1.3 Why Onboard Edge Intelligence? . . . 6
1.4 Scientific Problem . . . 6
1.5 Research Gap . . . 6
1.6 Research Questions . . . 7
1.7 Main Contributions . . . 7
1.8 Paper Organization . . . 7

2 Related Work **8**

2.1 UAV-Based Power-Line Inspection . . . 8
2.2 Insulator Detection and Condition Assessment . . . 8
2.3 Model Compression for Edge AI . . . 8
2.3.1 Structured Pruning . . . 8
2.3.2 Post-Pruning Recalibration . . . 9
2.3.3 Knowledge Distillation . . . 9
2.3.4 INT8 Quantization . . . 9
2.4 Knowledge Distillation for Object Detection . . . 9

2.4.1 Teacher–Student Architecture . . . 9
2.4.2 Fine-Grained Feature Imitation for Object Detectors . . . 10
2.4.3 Deep Structured Instance Graph Distillation . . . 10
2.4.4 Key-Area-Guided Knowledge Distillation for Transmission-Line Components . . . 11
2.4.5 Federated Knowledge Distillation for Insulator Defect Detection . . . 11
2.4.6 FIAEPI-KD for Missing-Insulator Detection . . . 12
2.4.7 Comparative Analysis of the Reviewed Studies . . . 12
2.5 Research Gap and Positioning . . . 13
2.6 Position of Knowledge Distillation in the Present Work . . . 13
3 Materials and Methods **14**
3.1 Overall UAV Inspection and Optimization Framework . . . 14
3.2 UAV Dataset Acquisition and Annotation . . . 14
3.2.1 Annotation Representation . . . 15
3.2.2 Experimental Dataset Preparation . . . 15
3.2.3 Experimental Data Split . . . 16
3.2.4 Dataset Statistics and Distribution . . . 16
3.2.5 Dataset Examples and Annotation Quality . . . 17
3.2.6 Independent EPRI Test Set . . . 17
3.3 Baseline Faster R-CNN Detector . . . 17
3.4 Main Contribution: Architecture-Aware Compression Framework . . . 18
3.5 Architecture-Aware Structured Pruning . . . 19
3.5.1 Taylor-Based Channel Importance . . . 19
3.5.2 Dependency-Aware Channel Removal . . . 19
3.5.3 Normalized Global Ranking . . . 19
3.5.4 Detector-Specific Protection Policy . . . 20
3.6 Post-Pruning Recalibration . . . 20
3.7 Teacher–Student Knowledge Distillation . . . 20
3.7.1 Teacher and Student Definition . . . 20
3.7.2 FPN Feature Distillation . . . 20
3.7.3 Distillation Objective . . . 20
3.7.4 Student Recovery Strategy . . . 20
3.8 INT8 Post-Training Quantization . . . 21
3.9 Validation Gates . . . 21
4 Experimental Setup **23**
4.1 Hardware and Software Environment . . . 23
4.2 Dataset and Evaluation Split . . . 24
4.3 Training Protocol . . . 24
4.3.1 Baseline Training . . . 24
4.3.2 Pruning and Recovery Training . . . 25
4.4 Compression Hyperparameters . . . 26
4.5 Evaluation Metrics . . . 27
4.5.1 Detection Metrics . . . 27
4.5.2 Efficiency Metrics . . . 28
4.6 Statistical and Reproducibility Protocol . . . 28
5 Results **30**
5.1 ResNet-18 Study: the Pipeline on a Second Architecture . . . 30
5.2 Baseline Faster R-CNN Inspection Performance . . . 33
5.3 Faster R-CNN Component Sensitivity Ablation . . . 34
5.4 Effect of Recalibration and Knowledge Distillation . . . 35
5.5 Recovery Recipe, Uniform Allocation and Zero-Out Test . . . 37
5.6 INT8 Quantization Results . . . 38
5.7 End-to-End Accuracy–Efficiency Trade-Off . . . 39
5.8 Error Analysis . . . 40

5.8.1 Failure Images . . . 43
5.9 Comparison with the Reference Experimental Protocol . . . 44
6 Discussion **46**
6.1 Main Scientific Findings . . . 46
6.2 Why Architecture-Aware Protection Matters . . . 46
6.3 Role of Knowledge Distillation . . . 46
6.4 Contribution of Quantization . . . 46
6.5 Implications for UAV Edge Deployment . . . 46
6.6 Comparison with the State of the Art . . . 46
6.7 Toward Embedded Hardware . . . 47
6.8 Limitations and Threats to Validity . . . 47
7 Conclusion and Future Work **48**
7.1 Conclusion . . . 48
7.2 Future Work . . . 48
References **50**
A Reproducibility Card **52**
A.1 Commands . . . 52
A.2 Settings of each stage . . . 53
A.3 Validation gates . . . 53
A.4 Checkpoints . . . 53
B Complete Results Ledger **54**
C Additional Baseline Analyses of the Teacher **55**
D Embedded Platforms and Integration Path **57**
D.1 What an embedded platform is . . . 57
D.2 The Raspberry Pi . . . 57
D.3 How the framework would be integrated . . . 58
D.4 Measurement protocol on the device . . . 59
E Register of Deviations and Open Items **61**

List of Figures

2.1 Original fine-grained feature imitation framework proposed by Wang et al. [13]. . . . 10
2.2 Original deep structured instance graph distillation framework proposed by Chen et al. [14]. . . 11
2.3 Original key-area-guided knowledge distillation framework proposed by Zhao et al. [15]. . . . 11
2.4 Original federated knowledge distillation framework for insulator defect detection proposed by Huang et al. [16]. . . . 12
2.5 Original FIAEPI-KD framework for missing-insulator detection proposed by Cui et al. [17]. . . 12

3.1 Overall UAV inspection and model-optimization pipeline. . . . 14
3.2 Distribution of annotated objects among the three experimental condition classes after label mapping. . . . 16
3.3 Baseline Faster R-CNN architecture. . . . 18
3.4 Prune–Recalibrate–Distill–Quantize checkpoints M0–M4. . . . 19
3.5 Teacher–student Faster R-CNN knowledge distillation. . . . 21
3.6 Stage-wise validation gates. . . . 22

5.1 ResNet-18 pipeline on Imagenette: accuracy, macro-F1, size, and parameter count after each stage. 31

5.2 ResNet-18: accuracy of the pruned network alone and after distillation, for each importance criterion. . . . 32
5.3 ResNet-18: distillation against cross-entropy recovery, with and without re-initialisation of the surviving weights. . . . 32
5.4 Qualitative baseline detections (M_0) on easy, difficult, distant, and damaged insulators. Ground truth in green, prediction in red. . . . 34
5.5 Retention versus parameter reduction. Circles: this study with the final recipe; squares (*): base recipe, 800 iterations; triangles: reference report. . . . 35
5.6 Retention at each stage (M_1 to M_3, final recipe) for the three main configurations. . . . 36
5.7 Distillation training curves (final recipe, 1,600 iterations; moving average over 7 logged points): total loss, task loss, and FPN feature-alignment loss. . . . 37
5.8 Retention of the aggressive student for each recovery change. The error bar is the standard deviation over three seeds of final recipe; the dashed line is the reference value. . . . 38
5.9 $AP_{50:95}$ versus model size. . . . 40
5.10 $AP_{50:95}$ versus CPU latency (cost session). The INT8 model is not shown because it was timed in a different session. . . . 40
5.11 Whole backbone: one insulator at M_0, M_1, M_2, M_3. . . . 41
5.12 Maximal pruning: one insulator at M_0, M_1, M_2, M_3. . . . 41
5.13 Recall per condition class at score 0.5 and IoU 0.5: the damage classes carry the loss of the compressed students. . . . 42
5.14 Failure cases of the maximal student. Left: M_0; right: student. (a) A flashover-damaged insulator that M_0 labels *Flashover damage* and the student labels *No issues*. (b) A broken insulator that M_0 labels *Broken* (0.93) and the student labels *No issues* (0.61) on almost the same box. (c) A broken insulator with a chipped cap that the student does not detect. (d) Healthy insulator units in a busy scene where the student outputs no box. . . . 43
5.15 AP per condition class for M_0 and the compressed students. . . . 44
5.16 Retention of every student of the final recipe computed against the original teacher M_0 and against the teacher fine-tuned with the same recipe, next to the reference report. . . . 45

C.1 Three-class confusion matrix of M_0 on the held-out subset. . . . 55
C.2 Precision, recall, and F1 of M_0 per class. . . . 55
C.3 Distribution of the IoU between matched predictions and ground-truth boxes for M_0. . . . 56

D.1 Onboard unit built around a Raspberry Pi 5 class board: the camera stream enters the board, the INT8 student runs on the CPU (or on an optional accelerator), and the result is a class and a box for each insulator. Temperature, throttling, and power are monitored from outside. . . . 58
D.2 Serialized size of the detector and of the compressed students (measured values). . . . 58
D.3 From the framework to a measured model on the device. The blue steps exist in this work; the orange steps are planned. . . . 59

List of Tables

1.1 Main inspection requirements: safety, coverage, resolution, repeatability, and early defect identification. . . . 6

2.1 Comparison of the five reviewed knowledge-distillation studies for object detection and insulator inspection. . . . 13

3.1 Distribution of annotations after the experimental class mapping. 15
3.2 Experimental dataset split used for Faster R-CNN training and evaluation. 16
3.3 Official IDID V1.2 dataset statistics and their role in the present experimental setup. 17

4.1 Hardware and software environment used for the experiments. 23
4.2 Operational dataset split used during the experiments. 24
4.3 Training and recovery protocol used for the compression experiments. 26
4.4 Compression hyperparameters and experimental factors. 27
4.5 Evaluation metrics and measurement protocol. 28

5.1 ResNet-18 on Imagenette, 128 × 128 inputs, pipeline run through the framework (15 recovery epochs). Accuracy and macro-F1 on the validation images; single run. 30
5.2 ResNet-18, macro-F1 (%) after distillation for four importance criteria and five parameter reductions; single run per cell. 31
5.3 ResNet-18, accuracy (%) after 6 recovery epochs, Taylor ranking; single run per cell. 32
5.4 Baseline detector M_0 on the 320-image held-out subset. 33
5.5 Pruning-policy ablation after recalibration and distillation (M_3, FP32). All rows use the final recipe (1,600 iterations). The three rows at 57.6% share the same reduction target. Parameters of the internal and residual rows are derived from their reduction. 34
5.6 $AP_{50:95}$ (retention) of the student at each stage (M_3: final recipe, 1,600 iterations). 35
5.7 Aggressive student (57.6% reduction) for two distillation budgets. 36
5.8 Recovery-recipe ablation on the aggressive student (57.6% reduction); one seed. Retention is relative to M_0. 37
5.9 INT8 results on the maximal student. Retention is relative to M_0. 38
5.10 Accuracy–efficiency summary (cost session: 800 × 1200 input, median of 20 runs, 4 threads, i5-6200U). 39
5.11 Error analysis on the 320 test images (score ≥ 0.5, IoU ≥ 0.5). Recall by condition class, size tertile and scene density, and counts of missed objects (FN) and false detections (FP) by cause. M_3 and M_4 are the second training of the maximal student and its INT8 version. 42
5.12 Retention ($AP_{50:95}$ relative to the respective teacher): reference report [27] versus this study. . 44

A.1 Hyperparameters of each stage as used in the reported experiments. 53
A.2 Teacher and main compressed students (final recipe). 53

B.1 All evaluated checkpoints. 54

D.1 Classes of embedded platforms and their fit with a Faster R-CNN detector. Hardware figures are those given by the manufacturers for the examples; the last column is our assessment, not a measurement. 57

E.1 Declared differences with the reference experimental protocol. 61
E.2 Declared differences for the ResNet-18 study (Section 5.1). 61

Chapter 1
Introduction

1.1 Criticality of Power-Line Inspection

Transmission networks cover large and geographically difficult areas and expose electrical components to weather, contamination, mechanical stress, vegetation, and progressive degradation. Insulators are critical because they provide the electrical isolation required between energized conductors and supporting structures. Damage can contribute to leakage, flashover, faults, outages, and safety risks. Inspection is therefore central to preventive maintenance, reliability, and early identification of abnormal conditions. Manual inspection remains costly, time-consuming, difficult to repeat systematically, and potentially hazardous for personnel.

Table 1.1: Main inspection requirements: safety, coverage, resolution, repeatability, and early defect identification.

Requirement	Description
Safety	Safe inspection without exposing personnel to high-voltage infrastructure.
Coverage	Sufficient coverage of power-line components during UAV inspection.
Resolution	Adequate image resolution for reliable defect identification.
Repeatability	Consistent inspection results across repeated flights and acquisitions.
Early defect identification	Early detection of visible defects to support maintenance decisions.

1.2 Why UAV-Based Inspection?

UAVs provide a flexible acquisition platform for long transmission corridors while reducing direct operator exposure to hazardous locations. High-resolution optical sensing can capture towers, conductors, insulators, and their surroundings, while repeated trajectories can support temporal monitoring. The UAV is treated here primarily as the acquisition platform whose image quality directly affects the downstream perception problem: small targets, clutter, occlusion, and variable illumination can make inspection difficult.

1.3 Why Onboard Edge Intelligence?

A cloud-centric workflow requires transmission of high-resolution imagery and therefore assumes sufficient bandwidth, connectivity, storage, and remote computing. In remote inspection environments, these assumptions introduce communication dependence and latency. Onboard inference moves part of the analysis closer to the data source, reducing the need for continuous high-volume transfer and enabling local detection before selected results are communicated. Recent UAV inspection research explicitly discusses the transition toward UAV-edge architectures and the need to balance accuracy, latency, model size, and energy consumption [18, 19].

1.4 Scientific Problem

The problem is formulated as accuracy–efficiency optimization under hardware constraints. The detector should preserve useful insulator localization and condition-assessment performance while reducing parameters, FLOPs, memory footprint, latency, and, ultimately, energy demand.

$$\max\ P(\theta) \quad \text{s.t.} \quad M(\theta) \le M_{\max},\ C(\theta) \le C_{\max},\ T(\theta) \le T_{\max},\ E(\theta) \le E_{\max}. \tag{1.1}$$

The present work directly measures predictive quality, parameters, computational cost, model size, and CPU latency. Energy is treated as a future hardware-validation variable unless direct measurements are available.

1.5 Research Gap

Existing UAV inspection work frequently prioritizes detection accuracy or custom lightweight architecture design, whereas compression studies often rely on generic networks and benchmarks. Less attention is devoted to systematically compressing a high-capacity two-stage detector while explicitly accounting for the heterogeneous sensitivity of its feature extractor, proposal network, and prediction head. The present work does not claim

that pruning, distillation, and quantization have never been combined. Its positioning is narrower: ordered integration, architecture-aware protection, detector-specific ablation, and application to high-resolution UAV insulator inspection. The central hypothesis is therefore: *where pruning is applied matters as much as how much is pruned.*

1.6 Research Questions

RQ1. Can a high-capacity Faster R-CNN detector be structurally compressed for edge-oriented UAV inspection while preserving useful detection and condition-assessment performance?
RQ2. Which detector components are robust to pruning, and which require explicit protection?
RQ3. Does gradient-informed Taylor scoring provide a more recoverable student than alternative ranking criteria under aggressive compression?
RQ4. To what extent can teacher–student knowledge distillation restore performance lost after structured pruning?
RQ5. What additional efficiency is obtained from INT8 post-training quantization after recovery?
RQ6. Which configuration provides the best accuracy–efficiency trade-off for deployment-oriented use?

1.7 Main Contributions

- An application-to-deployment formulation linking UAV acquisition, insulator localization, condition assessment, and edge-oriented model compression.
- An architecture-aware structured-pruning strategy combining Taylor importance, dependency-aware channel removal, normalized global ranking, and explicit protection of prediction-critical modules.
- A teacher–student recovery strategy using the original FP32 Faster R-CNN as frozen teacher and the pruned detector as trainable student with FPN feature transfer.
- An ordered Prune–Recalibrate–Distill–Quantize pipeline evaluated stage by stage using predictive and efficiency metrics.
- A component-wise ablation testing the hypothesis that useful compression depends on where capacity is removed, not only on nominal pruning ratio.

1.8 Paper Organization

Section 2 contains the preserved state-of-the-art related work. Section 3 presents the proposed materials and methods. Section 4 defines the experimental setup and reproducibility conditions. Section 5 reports baseline, ablation, recovery, quantization, and end-to-end results. Section 6 interprets the findings and limitations. Section 7 concludes the study and defines future work. The appendices document the repository, commands, hyperparameters, validation records, and declared deviations needed to reproduce the results.

Chapter 2
Related Work

2.1 UAV-Based Power-Line Inspection

Artificial Intelligence is increasingly applied in the energy sector to monitoring, diagnostics, forecasting, maintenance, optimization, and safety-related tasks. The state of the art reviewed in this work emphasizes that these applications must also satisfy requirements of reliability, robustness, computational efficiency, and practical deployment [1, 2, 3]. For transmission infrastructure, these requirements are particularly relevant because inspection covers large and geographically difficult areas and must preserve sufficient visual detail to identify critical components and defects.

The inspection process has progressively moved from manual observation toward sensor-assisted and UAV-based acquisition. UAVs offer a flexible platform for high-resolution visual inspection while reducing operator exposure to hazardous locations and allowing repeated observation of long transmission corridors. Recent UAV research further describes a transition toward edge-oriented processing, motivated by the bandwidth, communication, storage, and latency limitations of continuously transmitting high-resolution imagery to remote servers [18, 19]. Thus, UAV acquisition and edge intelligence are treated as complementary parts of the same inspection chain.

The imagery itself presents a demanding perception problem: insulators may be small in the scene, partly occluded, surrounded by clutter, affected by illumination changes, or affected by visually subtle damage. The detector must therefore maintain both semantic discrimination and accurate localization while its computational burden is reduced sufficiently for constrained execution.

2.2 Insulator Detection and Condition Assessment

Insulators provide electrical isolation between energized conductors and supporting structures, making their condition directly relevant to transmission reliability and safety. Inspection may need to distinguish healthy elements from damaged or abnormal conditions, while also localizing the relevant objects in high-resolution images.

A unified detection formulation can represent each object by its bounding box and condition label:

$$b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max}, c_i), \qquad c_i \in \{\text{Good}, \text{Broken}, \text{Flashover}\}. \tag{2.1}$$

One-stage detectors are attractive when low latency is the dominant requirement, whereas two-stage detectors provide an explicit region-proposal mechanism and strong localization reasoning. The present work retains Faster R-CNN as the high-capacity reference detector and studies how its computational structure can be reduced without treating all of its components as equally compressible. Recent UAV-edge work demonstrates that real-time inspection can be achieved through detector, software, and hardware co-design, but such systems use different detector families, datasets, and accelerators; their results are therefore used for context rather than as direct numerical benchmarks [19].

2.3 Model Compression for Edge AI

The state of the art developed for this project treats model compression as a combination of complementary mechanisms rather than as a single technique. Structured pruning changes the architecture, recalibration adapts internal normalization statistics after model modification, knowledge distillation transfers useful information from a high-capacity teacher to a lighter student, and low-precision quantization reduces numerical representation cost [4, 8, 10, 11].

2.3.1 Structured Pruning

Unstructured pruning removes individual weights and can obtain high sparsity, but irregular sparsity does not necessarily translate directly into efficient execution on conventional hardware. Structured pruning removes organized units such as channels or filters, producing a smaller dense architecture that is more naturally

exploitable by hardware and inference libraries [4, 6].

For modern detectors, pruning cannot be treated as independent deletion of arbitrary channels. Residual branches, normalization layers, concatenations, feature pyramids, proposal modules, and prediction heads create dependencies that must remain dimensionally consistent. DepGraph provides a dependency-aware representation for grouping operations that must be modified together, while global ranking allows candidate groups to be compared across the model [4]. EPIC further connects dependency-aware pruning to constrained-device inference, and AutoCompress investigates combinations of structured pruning strategies for high compression rates [5, 7].

2.3.2 Post-Pruning Recalibration

Pruning or other model modifications can alter activation distributions and make previously learned normalization statistics inappropriate for the modified network. Post-training BatchNorm recalibration addresses this effect by running representative data through the modified model and refreshing running statistics without full weight retraining [8]. This operation is conceptually different from confidence calibration, which concerns the agreement between predicted confidence and empirical correctness [9].

2.3.3 Knowledge Distillation

Knowledge Distillation (KD) transfers information from a higher-capacity teacher to a lighter student. The transferred information may come from final responses, intermediate representations, or relationships between instances and features [10]. In object detection, feature-level knowledge is particularly relevant because a detector must preserve localization and multi-scale representations in addition to class information. The five domain-relevant KD studies retained in this work are reviewed in detail in Section 2.4.

2.3.4 INT8 Quantization

Quantization reduces numerical precision and can decrease model storage and execution cost. The reviewed literature covers integer-arithmetic-only inference, scale and zero-point parameters, symmetric and asymmetric quantization, per-layer and per-channel quantization, post-training quantization, and quantization-aware training [11]. The practical gain is hardware dependent, as operator coverage and accelerator support determine which portions of a detector can actually execute in INT8 [12].

A generic affine quantization relation is:

$$q(x) = \text{clip}\left(\text{round}(x/s) + z, q_{\min}, q_{\max}\right). \tag{2.2}$$

For Faster R-CNN, this distinction is important because dynamic proposal filtering, RoI processing, and post-processing may not all map to static integer kernels. A deployment-oriented implementation should therefore report the supported INT8 subgraphs and the remaining FP32 operations explicitly.

2.4 Knowledge Distillation for Object Detection

Knowledge Distillation is a model compression and knowledge-transfer technique in which a lightweight student model learns from a higher-capacity teacher model. Instead of relying only on ground-truth annotations, the student is guided by information produced by the teacher. This additional supervision can help the student preserve useful representations and predictive capabilities while using a smaller or more efficient architecture [10].

Knowledge distillation is particularly relevant to Edge deployment because object detection models can be computationally expensive and difficult to execute on resource-constrained devices. Several studies have therefore investigated how knowledge can be transferred from a large detector to a lightweight one through outputs, intermediate features, structured relationships, or task-specific information.

The following five studies are retained strictly from the project's State of the Art because they represent the evolution of KD from general object-detector feature transfer toward transmission-line and insulator-specific knowledge transfer.

2.4.1 Teacher–Student Architecture

A conventional KD framework contains a teacher and a student. The teacher is usually larger, more accurate, and already trained, whereas the student is designed to be lighter and computationally more efficient. During

distillation, the teacher provides additional knowledge that guides student optimization.

The teacher is generally kept fixed while the student parameters are updated. Depending on the approach, the transferred information may come from final predictions, intermediate feature representations, or relationships between features and instances [10].

2.4.2 Fine-Grained Feature Imitation for Object Detectors

Wang et al. proposed *Distilling Object Detectors With Fine-Grained Feature Imitation* to address limitations of conventional knowledge distillation when applied to object detection [13]. The objective is to improve lightweight object detectors by transferring detailed spatial information from a high-capacity teacher. Because detection includes precise localization in addition to classification, the student should not simply reproduce final class predictions.

The method performs **fine-grained feature imitation**. Instead of forcing the student to reproduce teacher features uniformly over the whole feature map, it identifies informative locations around objects and gives greater importance to these regions. The resulting transfer focuses on feature responses near object-related locations, where the teacher contains useful information for distinguishing and localizing objects. The method was evaluated with object-detection models including Faster R-CNN and reports a reduction of the performance gap between lightweight students and their teachers [13].

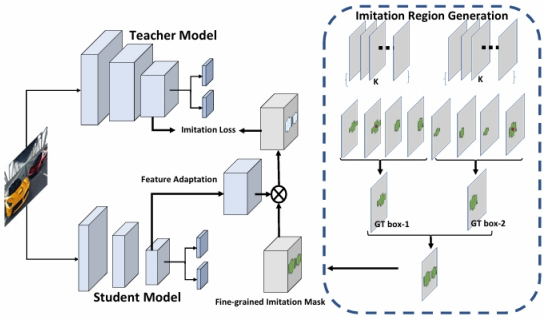


Figure 2.1: Original fine-grained feature imitation framework proposed by Wang et al. [13].

2.4.3 Deep Structured Instance Graph Distillation

Chen et al. proposed *Deep Structured Instance Graph for Distilling Object Detectors* to transfer structured relationships between instances from teacher detectors to student detectors [14]. The objective is to overcome the limitation of treating feature responses as independent elements. Object detection contains relationships between different instances, and these relationships can provide additional knowledge for the student.

The proposed method constructs a **structured instance graph**. Nodes correspond to instance proposal-level features, while edges represent relationships between these nodes. The teacher representation is used to construct the graph, which is transferred to the student. Adaptive background loss weighting reduces the influence of noisy background nodes, while background sample mining removes trivial relationships. The resulting representation transfers both local instance information and global relational information. The approach was evaluated with different teacher–student pairs and detector architectures, including Faster R-CNN with ResNet-FPN backbones [14].

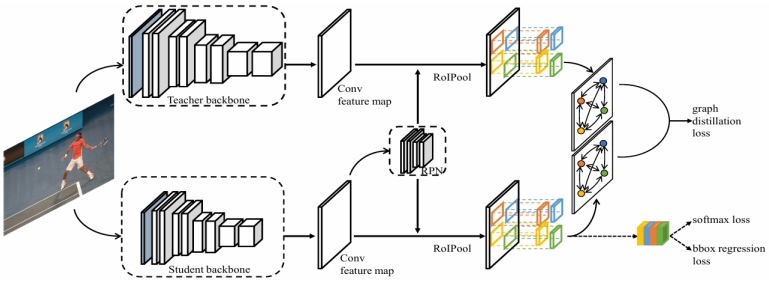


Figure 2.2: Original deep structured instance graph distillation framework proposed by Chen et al. [14].

2.4.4 Key-Area-Guided Knowledge Distillation for Transmission-Line Components

Zhao et al. proposed *Defect Detection Method for Key Area Guided Transmission Line Components Based on Knowledge Distillation* for defect detection of transmission-line components, including insulators [15]. The study addresses the limited number of defect images and the small spatial range of defect features. Its objective is to guide the student toward informative defect regions while also transferring contextual information related to the target and its surrounding transmission-line background.

The method introduces a **Prediction-Guided Weighting (PGW)** module to identify important foreground regions and restrict the distillation process to high-quality feature pixels. A mask constructed from high-quality feature locations becomes the focus of teacher-to-student feature transfer. A **GcBlock** module further captures relationships between target defects and their surrounding background. The authors evaluate the method with several object detectors, including Faster R-CNN [15].

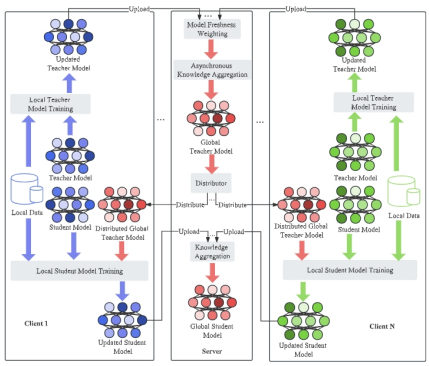


Figure 2.3: Original key-area-guided knowledge distillation framework proposed by Zhao et al. [15].

2.4.5 Federated Knowledge Distillation for Insulator Defect Detection

Huang et al. proposed *Federated Knowledge Distillation for Enhanced Insulator Defect Detection in Resource-Constrained Environments* to combine knowledge distillation with federated learning for lightweight insulator defect detection [16]. The purpose is to reduce computational complexity while limiting performance degradation associated with small student models, particularly in resource-constrained environments such as UAV-based inspection.

The proposed framework combines **knowledge distillation** with **federated learning** in a client-server structure. Clients train teacher and student models locally, while the server aggregates information received from different participants. The authors further introduce **asynchronous aggregation** and a **model freshness mechanism** to improve collaborative-training efficiency and reduce unnecessary aggregation of outdated or redundant models. The work therefore represents a different direction from conventional single-teacher offline distillation, embedding knowledge transfer in a federated collaborative-learning framework. The experimental framework reported by the authors uses YOLOv5 as teacher and a lightweight multilayer perceptron as student [16].

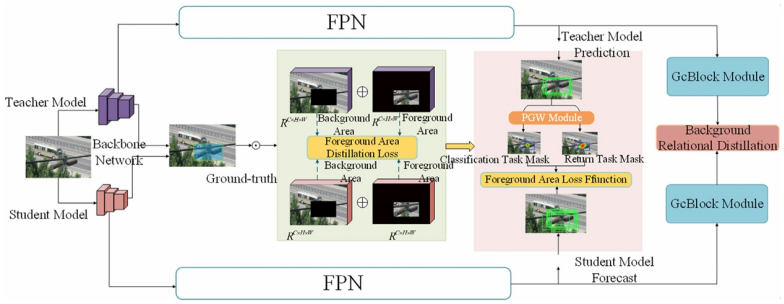


Figure 2.4: Original federated knowledge distillation framework for insulator defect detection proposed by Huang et al. [16].

2.4.6 FIAEPI-KD for Missing-Insulator Detection

Cui et al. proposed *FIAEPI-KD: A Novel Knowledge Distillation Approach for Precise Detection of Missing Insulators in Transmission Lines* to improve lightweight detection of missing insulators in complex transmission-line images [17]. The study focuses on small or weakly represented targets and aims to transfer detailed multi-scale and edge-related information from a teacher to a lightweight student.

The FIAEPI-KD framework combines two mechanisms: **Feature Indicator Attention (FIA)**, which identifies and emphasizes informative feature regions across different scales, and **Edge Preservation Index (EPI)**, which introduces edge-related information into the distillation process to preserve detailed target boundaries. The framework employs multi-scale representations through a ResNet backbone and Feature Pyramid Network. Experiments include Faster R-CNN and other detector architectures [17].

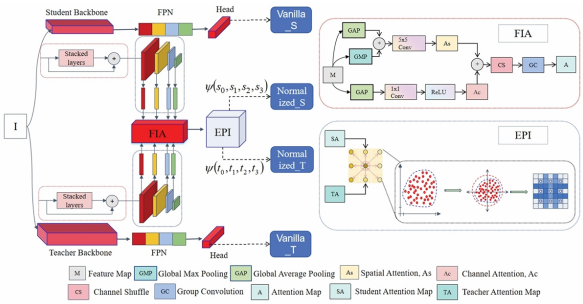


Figure 2.5: Original FIAEPI-KD framework for missing-insulator detection proposed by Cui et al. [17].

2.4.7 Comparative Analysis of the Reviewed Studies

The five reviewed studies illustrate several directions in the development of knowledge distillation for visual detection. Wang et al. concentrate on **where** feature knowledge should be transferred by emphasizing informative locations around detected objects. Chen et al. extend this concept by transferring **relationships between instances** through a structured graph. Zhao et al. adapt feature distillation to transmission-line inspection by explicitly focusing on **key defect regions and contextual relationships**. Huang et al. introduce a different dimension by combining distillation with **federated learning**, targeting collaborative training and resource-constrained insulator inspection. Finally, Cui et al. focus specifically on missing-insulator detection and incorporate **attention-guided and edge-preserving information** into the distillation process [13, 14, 15, 16, 17].

These methods demonstrate that knowledge transferred during object-detection distillation is not limited to final prediction outputs. It may include spatially important feature responses, structured instance relationships, defect-specific regions, contextual information, and edge representations.

Table 2.1: Comparison of the five reviewed knowledge-distillation studies for object detection and insulator inspection.

Study	Main objective	Main distillation principle
Wang et al. (2019)	Improve lightweight object detectors	Fine-grained imitation of teacher features at informative object-related locations.

2.5 Research Gap and Positioning

The reviewed literature reveals two complementary trends. UAV power-inspection studies demonstrate the value of high-resolution visual acquisition and increasingly edge-oriented analysis, while model-compression studies provide mechanisms for reducing computational and memory requirements. At the same time, generic pruning, recalibration, KD, and quantization cannot be assumed to affect every part of a two-stage detector equally.

The research gap is therefore stated conservatively. Existing UAV inspection work often prioritizes detection accuracy, whereas compression research frequently relies on generic benchmarks or does not systematically characterize the heterogeneous sensitivity of a high-capacity detector's feature extractor, proposal network, and prediction head. Less attention has been devoted to an ordered compression study for a high-capacity two-stage detector applied specifically to UAV-based power-line inspection while explicitly considering where the pruning budget should be spent.

The intended positioning is not the claim that pruning, knowledge distillation, and quantization have never been combined. Rather, the contribution lies in their **ordered integration**, **architecture-aware protection**, **detector-specific ablation**, and application to high-resolution UAV insulator inspection.

2.6 Position of Knowledge Distillation in the Present Work

The five reviewed KD studies are retained as state-of-the-art references and are not presented as methods directly implemented in the experimental pipeline. They establish why teacher–student transfer can preserve useful detector information and illustrate several alternatives: fine-grained spatial imitation, structured instance relations, key-region guidance, federated knowledge transfer, and edge-aware multi-scale distillation.

For the present Faster R-CNN compression pipeline, the teacher is the original FP32 detector and remains frozen. The student is the structurally pruned and recalibrated detector. Feature-level knowledge is transferred through the FPN hierarchy at levels $P2$–$P5$. The objective used by the experimental methodology is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \beta \, \text{MSE}(F_S, F_T), \tag{2.3}$$

with

$$\beta = 0.5. \tag{2.4}$$

This distinction is essential: the literature methods are not interchangeable with the project's compression pipeline. Pruning constructs the smaller architecture, recalibration adapts normalization statistics, KD recovers useful information from the frozen teacher, and INT8 quantization is applied after FP32 recovery. The full sequence is therefore:

$$\text{Original FP32} \rightarrow \text{Structured Pruning} \rightarrow \text{Recalibration} \rightarrow \text{Knowledge Distillation} \rightarrow \text{INT8 Quantization}. \tag{2.5}$$

This positioning leads directly to the central scientific hypothesis of the paper: *where pruning is applied matters as much as how much is pruned*. The following methodology and ablation experiments are designed to test that hypothesis.

Chapter 3
Materials and Methods

3.1 Overall UAV Inspection and Optimization Framework

The complete application-to-deployment chain is:

$$\begin{aligned}&\text{Power-line scene} \rightarrow \text{UAV acquisition} \rightarrow \text{Dataset} \rightarrow \text{Faster R-CNN}\\ &\rightarrow \text{Prune} \rightarrow \text{Recalibrate} \rightarrow \text{Distill} \rightarrow \text{INT8} \rightarrow \text{Edge-oriented model.}\end{aligned} \tag{3.1}$$

The model checkpoints are defined as M_0 original FP32 teacher, M_1 structurally pruned student, M_2 recalibrated student, M_3 recovered FP32 student, and M_4 final partially INT8 deployment-oriented student.

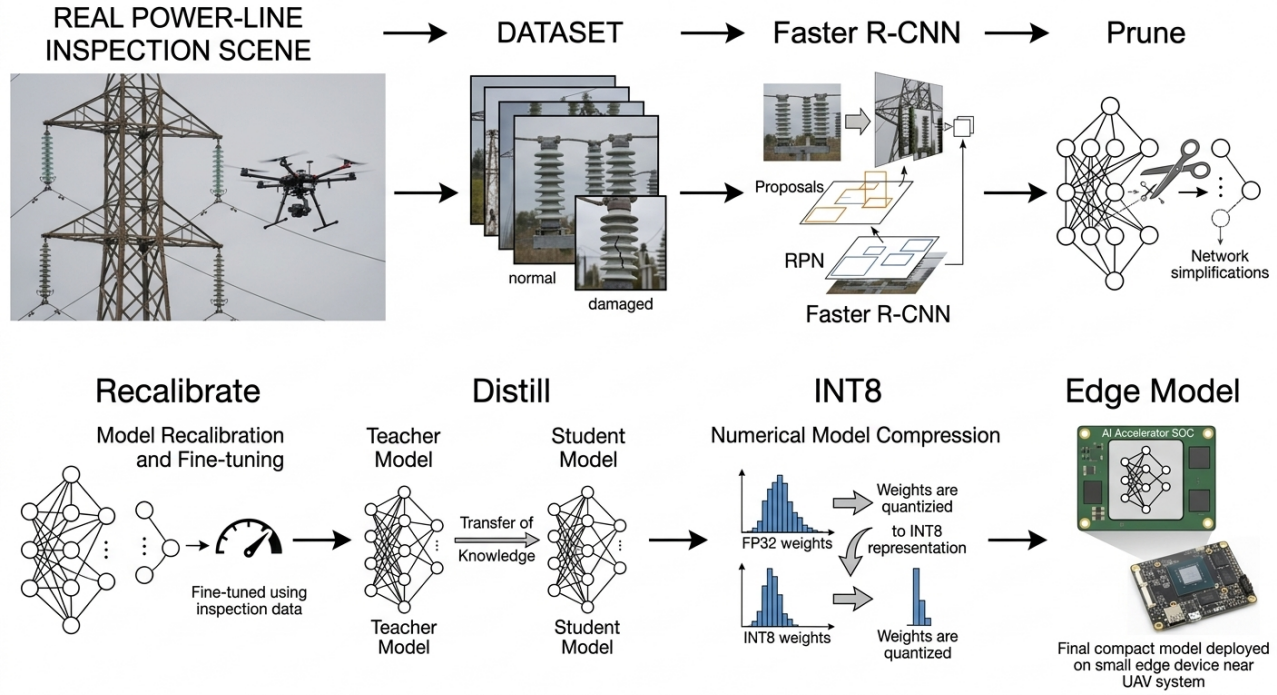


Figure 3.1: Overall UAV inspection and model-optimization pipeline.

3.2 UAV Dataset Acquisition and Annotation

The experimental dataset used in this project is derived from the *EPRI Insulator Defect Image Dataset (IDID) V1.2*. The dataset was selected as the basis for the development and evaluation of the proposed Faster R-CNN based inspection system for UAV-assisted power infrastructure inspection.

The official IDID V1.2 dataset contains publicly available training images together with an independent test set held by EPRI. The official training release contains 1,596 images and 7,568 labeled assets, while the EPRI-held test set contains 88 images and 403 labeled assets. These official dataset statistics are reported here to describe the source dataset.

For the present experiments, however, the locally available training corpus contains 1,600 images. The annotations were processed locally and converted into the representation required by the Faster R-CNN training pipeline. Therefore, the statistics reported in the remainder of this section refer to the actual dataset used in the present experiments rather than directly to the official dataset summary.

3.2.1 Annotation Representation

The original IDID annotations describe the location and condition of insulator-related objects using bounding boxes. For each annotated object i, the bounding-box representation used in the experimental pipeline can be expressed as

$$b_i = (x_{\min}, y_{\min}, x_{\max}, y_{\max}, c_i), \tag{3.2}$$

where $x_{\min}$ and $y_{\min}$ denote the upper-left corner of the bounding box, $x_{\max}$ and $y_{\max}$ denote the lower-right corner, and c_i represents the corresponding condition class.

The original annotations were provided in the dataset JSON format using bounding boxes represented by the coordinates (x, y, w, h). These annotations were converted to the coordinate representation required by the object-detection pipeline while preserving the original object locations.

The source annotations contain several condition-related categories. For the present experiments, the annotation labels were explicitly mapped into three detection classes:

- **No issues**,
- **Broken**,
- **Flashover damage**.

The categories corresponding to normal or non-damaged insulator conditions were grouped into the *No issues* class. The *Broken* and *Flashover damage* conditions were retained as separate defect classes. This mapping was applied explicitly during dataset preparation and was kept unchanged for the baseline Faster R-CNN experiment and the subsequent compression stages.

3.2.2 Experimental Dataset Preparation

The locally available dataset contains 1,600 images and a corresponding `labels_v1.2.json` annotation file. After loading the annotations, the source condition labels were mapped to the three experimental classes described above.

Following this mapping, the complete local dataset contains 17,108 bounding-box annotations distributed as follows:

Table 3.1: Distribution of annotations after the experimental class mapping.

Experimental class	**Number of annotations**
No issues	13,364
Broken	1,180
Flashover damage	2,564
Total	**17,108**

The resulting class distribution is visualized in Figure 3.2.

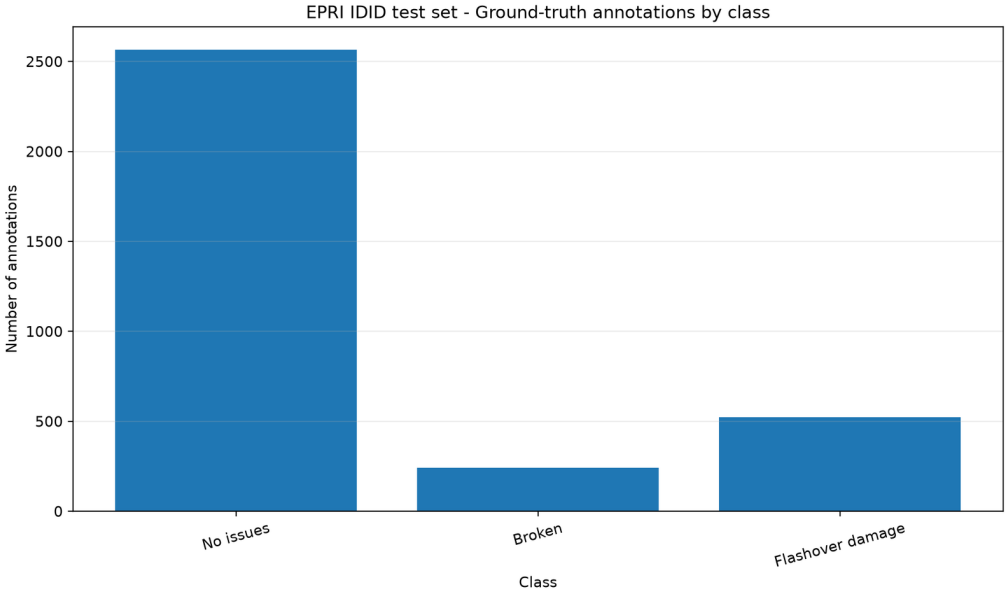


Figure 3.2: Distribution of annotated objects among the three experimental condition classes after label mapping.

3.2.3 Experimental Data Split

The 1,600 locally available images were divided into a training subset and a held-out evaluation subset using a fixed random state of 42. A test fraction of 20% was used for the evaluation subset:

$$D = D_{\text{train}} \cup D_{\text{eval}}, \tag{3.3}$$

with

$$D_{\text{train}} \cap D_{\text{eval}} = \varnothing. \tag{3.4}$$

The resulting split contains 1,280 training images and 320 held-out evaluation images. The corresponding annotation counts are 13,777 and 3,331, respectively.

Table 3.2: Experimental dataset split used for Faster R-CNN training and evaluation.

Subset	**Images**	**Annotations**
Training	1,280	13,777
Evaluation	320	3,331
Total	**1,600**	**17,108**

The same 320-image held-out evaluation subset is used as the primary evaluation population for the baseline M0 Faster R-CNN experiment and is maintained as the reference evaluation population for the subsequent compression stages.

3.2.4 Dataset Statistics and Distribution

The experimental dataset presents a substantially imbalanced distribution among the three condition classes. The *No issues* class contains the largest number of annotations, followed by *Flashover damage* and *Broken* conditions.

This distribution is relevant to the interpretation of the object detection results because the number of available examples is not uniform across the three classes. In particular, the lower number of broken insulator annotations compared with normal-condition annotations can affect the amount of visual variation available to the detector during training.

The distribution of annotated objects provides an important characterization of the experimental dataset

before model training. Additional statistics related to image dimensions and bounding-box distributions are reported through the dataset analysis figures below.

3.2.5 Dataset Examples and Annotation Quality

Representative ground-truth examples were extracted from the experimental dataset to visually inspect the object annotations and the correspondence between the image content and the assigned condition classes.

The examples provide a qualitative representation of the visual variability present in the dataset and of the bounding-box annotations used to train the Faster R-CNN detector.

3.2.6 Independent EPRI Test Set

The official IDID V1.2 release also contains an independent EPRI-held test set comprising 88 images and 403 labeled assets. This set is kept separate from the locally processed 1,600-image corpus and from the 320-image held-out evaluation subset used for the primary quantitative evaluation.

The EPRI-held test set is therefore not merged with the local training or evaluation data. When used in the present project, it serves as an additional source for qualitative inspection and inference rather than as a replacement for the primary quantitative evaluation split.

Table 3.3: Official IDID V1.2 dataset statistics and their role in the present experimental setup.

Category	**Official Train**	**EPRI Test**
Images	1,596	88
Labeled assets	7,568	403
Insulator strings	2,004	89
Good insulator shell	1,788	103
Broken insulator shell	2,636	147
Flashover damage insulator shell	1,140	64

The distinction between the official dataset statistics and the locally processed experimental dataset is maintained throughout the report. Official IDID statistics describe the source dataset, whereas the 1,600-image corpus, its three-class mapping, and the 1,280/320 split define the data actually used for the present Faster R-CNN experiments.

3.3 Baseline Faster R-CNN Detector

The reference detector is Faster R-CNN with a ResNet-50 backbone, FPN multi-scale features P2–P5, an RPN for region proposals, and an RoI head for classification and bounding-box regression. Only the components relevant to the compression policy are emphasized. The target is a high-capacity two-stage detector whose structure can be reduced without redesigning the full perception problem.

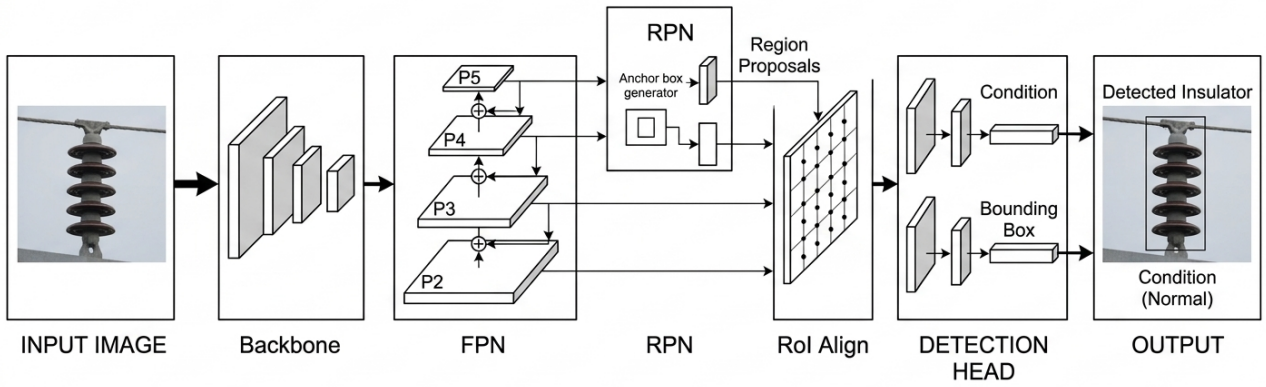


Figure 3.3: Baseline Faster R-CNN architecture.

3.4 Main Contribution: Architecture-Aware Compression Framework

The ordered checkpoint sequence is:

$$M_0 \xrightarrow{\text{structured pruning}} M_1 \xrightarrow{\text{recalibration}} M_2 \xrightarrow{\text{knowledge distillation}} M_3 \xrightarrow{\text{INT8 PTQ}} M_4. \tag{3.5}$$

Pruning changes the architecture, recalibration restores valid normalization statistics, distillation recovers predictive capability, and quantization reduces numerical precision after the recovered FP32 model converges.

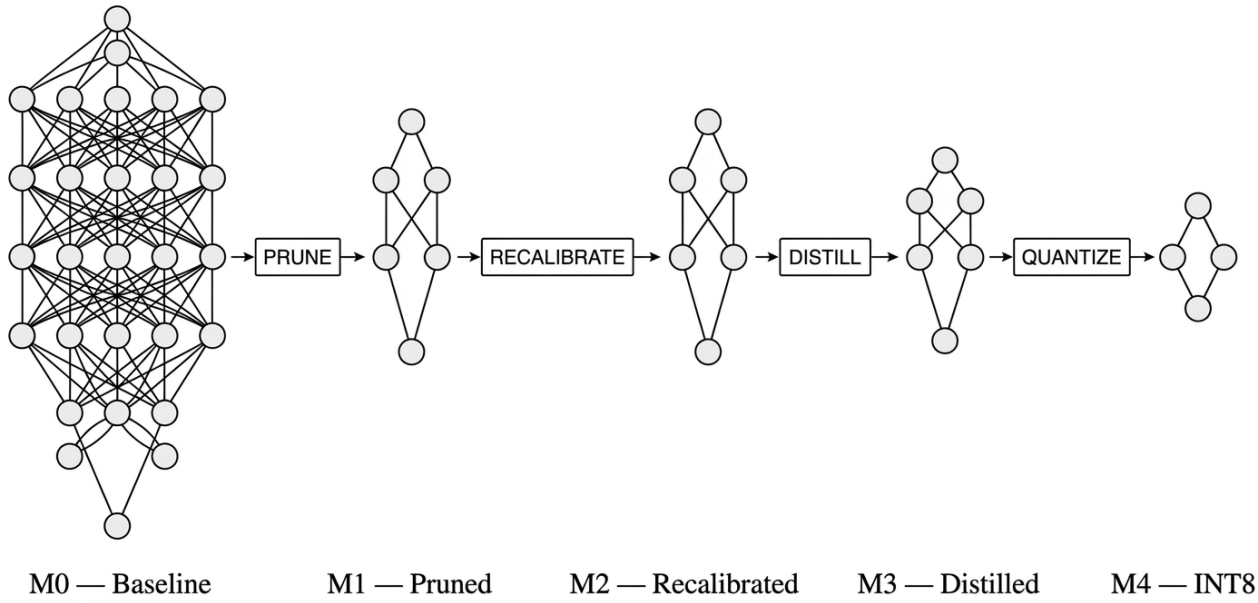


Figure 3.4: Prune–Recalibrate–Distill–Quantize checkpoints M0–M4.

3.5 Architecture-Aware Structured Pruning

The pruning criterion uses channel-level Taylor importance:

$$I_c = \left| W_c \frac{\partial L}{\partial W_c} \right|. \tag{3.6}$$

For dependency group G:

$$I_{G,c} = \sum_{l \in G} I_{l,c}, \qquad \widetilde{I}_{G,c} = \frac{I_{G,c} - I_{G,\min}}{I_{G,\max} - I_{G,\min} + \epsilon}. \tag{3.7}$$

An optional group aggressiveness coefficient is applied as $I_{\text{final}} = w_G \widetilde{I}_{G,c}$. Importance is accumulated on a calibration subset without optimizer updates, then low-scoring channels are removed with dependency-aware structural surgery.

3.5.1 Taylor-Based Channel Importance

Taylor scoring approximates the local first-order contribution of a channel to the loss and is evaluated across many representative images. The resulting channel scores are aggregated before any architecture modification occurs.

3.5.2 Dependency-Aware Channel Removal

Structural removal must propagate through dependent convolutions, biases, normalization layers, and residual branches. For the Detectron2 Faster R-CNN, dependencies are resolved by an in-house dependency graph that groups the convolutions, normalization layers, and residual connections of each ResNet stage and the FPN branches, following the principle of DepGraph [4]. The detector-specific protection policy remains a separate scientific design decision.

3.5.3 Normalized Global Ranking

Scores from heterogeneous groups are normalized before a global ranking. The minimum retained-channel constraint prevents invalid or excessively narrow layers. Compression is therefore governed by an explicit global budget plus architecture-aware group exposure.

3.5.4 Detector-Specific Protection Policy

The stem, RPN, and RoI prediction head are treated as protected or weakly pruned because they are functionally sensitive to representation quality, proposal generation, and final localization/classification. Most structural compression is allocated to the feature-extraction backbone and compatible FPN components.

$$\text{Sensitivity(Backbone)} \neq \text{Sensitivity(RPN)} \neq \text{Sensitivity(RoI Head)}. \tag{3.8}$$

3.6 Post-Pruning Recalibration

Pruning changes activation distributions and can invalidate normalization statistics. A representative calibration subset is passed through the pruned network in forward-only mode to refresh affected running statistics. No parameter optimization is performed in this stage, and the refreshed statistics are then frozen.

$$M_1 + D_{\text{cal}} \xrightarrow{\text{forward only}} M_2. \tag{3.9}$$

3.7 Teacher–Student Knowledge Distillation

3.7.1 Teacher and Student Definition

The teacher is the original FP32 detector and is frozen during distillation. The student is the structurally pruned and recalibrated model:

$$\text{Teacher} = M_0, \qquad \text{Student} = M_2. \tag{3.10}$$

3.7.2 FPN Feature Distillation

Knowledge is transferred at FPN levels P2–P5. Let F_T^p and F_S^p denote teacher and student features at level p:

$$L_{\text{FPN}} = \frac{1}{|P|} \sum_{p \in \{P2,P3,P4,P5\}} \left\| F_T^p - F_S^p \right\|^2. \tag{3.11}$$

3.7.3 Distillation Objective

$$L_{\text{total}} = L_{\text{det}} + \lambda_{\text{feat}} L_{\text{FPN}}. \tag{3.12}$$

In all experiments $\lambda_{\text{feat}} = \beta = 0.5$ (Appendix A.2).

3.7.4 Student Recovery Strategy

The student starts from M_2 and is optimized while the teacher remains fixed. The same inspection samples provide both the task-learning signal and the feature-level recovery signal.

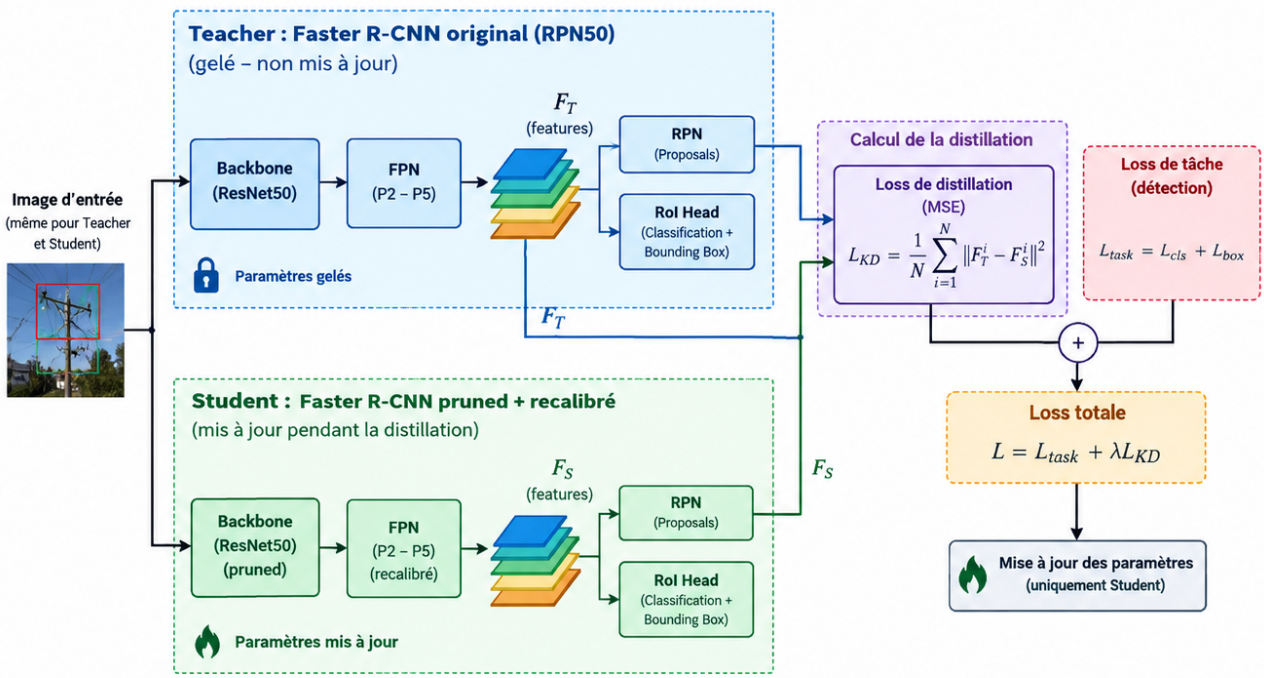


Figure 3.5: Teacher–student Faster R-CNN knowledge distillation.

3.8 INT8 Post-Training Quantization

Quantization is applied only after the recovered FP32 student converges. A generic affine relation is:

$$q(x) = \text{clip}\left(\text{round}(x/s) + z, q_{\min}, q_{\max}\right). \tag{3.13}$$

The workflow includes graph preparation, observer calibration, supported Conv–BN–ReLU fusion, conversion, and post-conversion validation. Because Faster R-CNN contains dynamic operations, the final model may be only partially INT8; dynamic post-processing that cannot be safely converted remains FP32.

3.9 Validation Gates

1. **Shape/forward gate:** structural changes preserve valid tensor dimensions and forward execution.
2. **Zero-ratio equivalence gate:** a zero pruning ratio reproduces original output behavior within the defined tolerance.
3. **Finiteness gate:** NaN/Inf activations, losses, or outputs cause immediate rejection.
4. **Accounting gate:** parameter and computational reductions are independently verified.
5. **Task-performance gate:** every checkpoint is evaluated on the same held-out protocol.

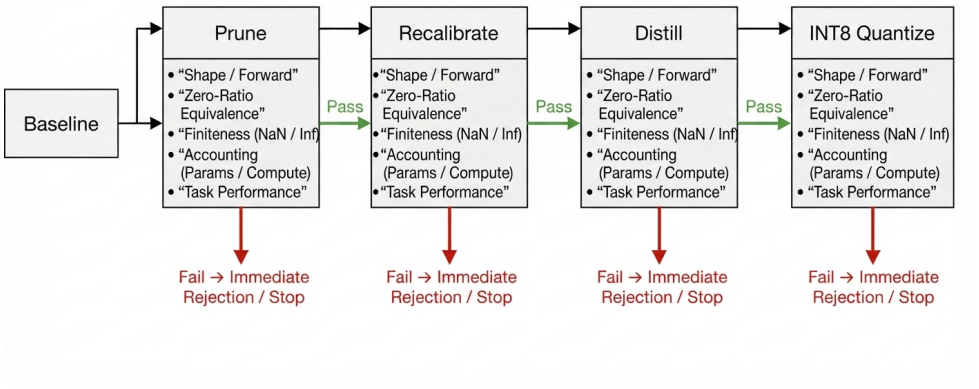


Figure 3.6: Stage-wise validation gates.

Chapter 4
Experimental Setup

This chapter defines the experimental protocol used to evaluate the proposed compression framework on the target Faster R-CNN detector for UAV-based insulator inspection. The objective is to make the comparisons between the baseline, pruned, recovered, and quantized models reproducible and scientifically interpretable.

Particular attention is given to the separation between model accuracy and deployment efficiency. A compression strategy is considered useful only when the reduction in computational and storage requirements is evaluated together with its effect on detection performance.

The experiments follow the ordered optimization pipeline introduced in the previous chapter:

Original FP32 → Structured Pruning → Recalibration → Knowledge Distillation → INT8 PTQ

Five principal checkpoints are considered throughout the experimental protocol:

- **M0**: original FP32 Faster R-CNN, used as the reference detector and frozen teacher;
- **M1**: structurally pruned detector, evaluated immediately after channel removal;
- **M2**: pruned detector after normalization-statistics recalibration;
- **M3**: recovered FP32 student after knowledge distillation;
- **M4**: final detector after INT8 post-training quantization.

The experimental design therefore evaluates both the final compressed model and the contribution of each intermediate optimization stage.

4.1 Hardware and Software Environment

All experiments are performed under a controlled computational environment. Hardware and software configurations are documented because they directly affect training behaviour, memory consumption, and deployment-oriented measurements.

The target detector is the Faster R-CNN architecture with a ResNet-50 backbone and Feature Pyramid Network (FPN). The architecture is configured according to the experimental model defined in the preceding methodology chapter.

The available development machine is an HP EliteBook-class system equipped with an Intel Core i5-6200U processor and 8 GB of installed RAM. The experiments are conducted under a 64-bit Windows operating system.

The exact software versions used for each experiment are recorded together with the corresponding experiment logs. Numerical model-complexity and performance measurements are intentionally reported in Chapter 5 rather than being mixed with the experimental setup.

Table 4.1: Hardware and software environment used for the experiments.

Element	Configuration
Processor	Intel(R) Core(TM) i5-6200U CPU @ 2.30 GHz
Installed RAM	8.00 GB (7.88 GB usable)
System type	64-bit operating system, x64-based processor
Operating system	Microsoft Windows 10 Pro
Windows version	21H2
OS build	19044.2965
Python	Python 3.11 environment used for the compression framework
Deep learning framework	PyTorch 2.14 (CPU build)
Computer vision framework	TorchVision
Object detection framework	Detectron2
Pruning framework	In-house dependency-aware structural pruning (DepGraph principle)

For deployment-oriented latency measurements, the CPU configuration and number of execution threads are kept fixed across the compared checkpoints. Warm-up iterations are performed before timed inference runs in order to reduce the influence of initialization overhead.

The numerical latency, throughput, model size, parameter count, and computational-cost measurements are reported in Chapter 5 after the corresponding experiments have been completed.

4.2 Dataset and Evaluation Split

The experiments use the EPRI Insulator Defect Image Dataset (IDID) version 1.2 described in Chapter 3. The dataset preparation, annotation processing, class mapping, and complete statistical characterization are intentionally presented in Chapter 3 and are not repeated here.

For the experimental evaluation, a fixed training/evaluation partition is used throughout the complete compression pipeline. The split is generated using a fixed random state and is preserved for all checkpoints from M0 to M4.

The evaluation subset remains unchanged when comparing the original, pruned, recalibrated, distilled, and quantized models. This ensures that differences in measured detection performance are attributable to the model optimization stages rather than to changes in the evaluation population.

The official external IDID test images are not mixed with the primary training/evaluation split. They are treated separately when qualitative inference or additional visual inspection is performed.

Table 4.2: Operational dataset split used during the experiments.

Element	**Experimental protocol**
Dataset	EPRI Insulator Defect Image Dataset (IDID) v1.2
Dataset role	Training and held-out evaluation of the detector
Primary split	Fixed training/evaluation split defined in Chapter 3
Split ratio	80% training / 20% held-out evaluation
Random state	42
Evaluation set	Fixed throughout M0–M4
Calibration data	Representative images separated from the final evaluation data
Official external test images	Kept separate from the primary COCO evaluation

The fixed split prevents evaluation leakage between the compression stages. In particular, images used for pruning-importance estimation, normalization recalibration, or quantization calibration are not used as the final held-out evaluation data.

4.3 Training Protocol

The training protocol is defined separately for the baseline detector and for the recovery stage. This distinction is important because structured pruning modifies the network architecture before knowledge distillation is applied.

The same evaluation definitions and held-out evaluation subset are preserved throughout the complete experimental sequence.

4.3.1 Baseline Training

The baseline Faster R-CNN detector is trained using the prepared IDID dataset and the preprocessing and augmentation strategy defined in the experimental configuration.

The baseline checkpoint M0 constitutes the reference model against which the subsequent compression stages are evaluated.

The following variables are fixed and recorded in the experiment configuration:

- input image preprocessing and resolution;
- batch size;
- optimizer;
- initial learning rate;

- learning-rate scheduler;
- number of training iterations;
- data-augmentation strategy;
- random seed;
- model initialization.

For the current baseline experiment, the training configuration is expressed in terms of training iterations rather than epochs. The M0 experiment uses the corresponding fixed training budget recorded in the training notebook.

The resulting model is saved as an FP32 checkpoint and subsequently used as the reference detector and frozen teacher for the compression experiments.

4.3.2 Pruning and Recovery Training

Structured pruning itself does not involve optimizer updates. Channel importance is estimated from gradient information accumulated over a representative subset while the original model weights remain unchanged.

After the importance scores have been computed, the selected channels are removed according to the dependency-aware pruning procedure described in Chapter 3.

The resulting network is evaluated immediately after structural modification. This produces checkpoint M1 and measures the direct effect of structural channel removal before any recovery operation.

The M2 checkpoint is then obtained through normalization-statistics recalibration. During this stage, the recalibrated statistics are obtained using forward-only passes over a calibration subset. No optimizer update is performed.

Knowledge distillation is subsequently applied to the pruned and recalibrated student. The original M0 Faster R-CNN remains frozen, while only the student parameters are updated.

For the detector, the implemented distillation mechanism operates on the multi-scale FPN representations:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \beta \mathcal{L}_{\text{FPN}},$$

with

$$\mathcal{L}_{\text{FPN}} = \frac{1}{|\mathcal{P}|} \sum_{p \in \{P2,P3,P4,P5\}} \|F_T^p - F_S^p\|_2^2 ,$$

where F_T^p and F_S^p denote the teacher and student feature maps at FPN level p, respectively.

The exact distillation coefficient used in the final implementation is reported with the corresponding experimental configuration.

The surviving weights after pruning are retained as the initialization of the student. The pruned network is therefore not reinitialized from scratch. This preserves the information already encoded in the remaining structure.

Table 4.3: Training and recovery protocol used for the compression experiments.

Element	**Protocol**
Baseline model	Faster R-CNN with ResNet-50-FPN
Baseline checkpoint	M0
Initialization	Configured pretrained/project initialization
Input preprocessing	Fixed experimental preprocessing
Training budget	Fixed number of training iterations
Random seed	Fixed experimental seed
Pruning	One-shot structured channel pruning
Importance estimation	Taylor-based gradient importance
M1	Structurally pruned FP32 detector
Recalibration	Forward-only normalization-statistics update
M2	Pruned and recalibrated detector
Teacher	Original FP32 M0 detector, frozen
Student	Pruned and recalibrated detector
Distillation target	FPN representations P2–P5
Student initialization	Surviving weights after pruning
M3	Recovered FP32 student
Final optimization	INT8 post-training quantization
M4	Final quantized detector

4.4 Compression Hyperparameters

The compression experiments investigate both the pruning criterion and the distribution of the pruning budget across the detector.

The principal structured-pruning criterion is based on first-order Taylor importance:

$$I_c = \left| W_c \frac{\partial \mathcal{L}}{\partial W_c} \right|,$$

where W_c denotes the parameters associated with output channel c.

Channel importance is accumulated over a representative subset without performing optimizer updates. The resulting importance values are normalized within the relevant dependency groups before the global ranking procedure.

The pruning process incorporates dependency-aware structural removal. Residual branches and other coupled tensors are therefore modified consistently so that the resulting network remains executable.

A minimum retained-channel constraint is imposed according to the final implementation. Prediction-critical detector components are protected or subjected to reduced pruning aggressiveness according to the architecture-aware policy described in Chapter 3.

The experiments therefore distinguish between the total pruning ratio and the location of the pruning budget inside the detector.

Table 4.4: Compression hyperparameters and experimental factors.

Parameter	**Experimental setting**
Pruning criterion	Taylor gradient-based importance
Importance formulation	$\lvert W \cdot \partial \mathcal{L} / \partial W \rvert$
Pruning granularity	Structured channel/filter pruning
Ranking	Normalized global ranking
Dependency handling	Dependency-aware structural pruning
Minimum retained channels	Defined by the final pruning implementation
Global pruning ratio	Experimental pruning ratio(s)
Protected components	Detector components defined by the protection policy
Pruning allocation	Uniform and architecture-aware configurations
Calibration subset	Representative images disjoint from final evaluation
Knowledge distillation	FPN feature distillation
Distillation target	FPN levels P2–P5
Quantization	Static INT8 post-training quantization
Quantization calibration	Calibration subset separated from final evaluation

The final numerical values of pruning ratios, minimum channel constraints, distillation coefficients, and quantization parameters are taken directly from the corresponding experiment logs and implementation configuration. They are not inferred from reference experiments.

4.5 Evaluation Metrics

The evaluation protocol separates predictive performance from computational efficiency. This distinction is necessary because a reduction in model size does not automatically imply an equivalent reduction in inference cost, while a computational improvement is not useful if the detector loses excessive predictive capability.

4.5.1 Detection Metrics

The detector is evaluated using the standard object-detection metrics consistent with the COCO evaluation protocol:

- $AP_{50:95}$;
- AP_{50};
- AP_{75};
- precision;
- recall;
- F1-score.

The principal COCO metrics are computed using the same evaluation subset for all model checkpoints.

Precision and recall are defined as

$$P = \frac{TP}{TP + FP},$$

and

$$R = \frac{TP}{TP + FN}.$$

The F1-score is given by

$$F_1 = \frac{2PR}{P + R}.$$

When precision, recall, and F1-score are reported, the detection threshold and matching protocol are kept identical across the compared checkpoints.

No performance value is introduced in this chapter. The numerical results for M0–M4 are presented and analysed in Chapter 5.

4.5.2 Efficiency Metrics

The efficiency analysis includes the following measurements:

- number of trainable parameters;
- computational cost measured using the selected complexity metric;
- on-disk model size;
- CPU inference latency;
- throughput, when measured consistently.

The same measurement procedure is applied to all checkpoints whenever the corresponding metric is technically applicable.

The following derived quantities are used to summarize the compression benefit:

$$\text{Compression Ratio} = \frac{\text{Size}_{\text{baseline}}}{\text{Size}_{\text{compressed}}},$$

$$\text{Speed-up} = \frac{\text{Latency}_{\text{baseline}}}{\text{Latency}_{\text{compressed}}},$$

and

$$\text{Performance Retention} = 100 \times \frac{\text{AP}_{\text{compressed}}}{\text{AP}_{\text{teacher}}}.$$

These derived quantities are used only after the underlying measurements have been obtained from the corresponding experiments.

Table 4.5: Evaluation metrics and measurement protocol.

Metric	**Evaluation protocol**
$AP_{50:95}$	COCO-style average precision over IoU thresholds 0.50–0.95
AP_{50}	Average precision at IoU = 0.50
AP_{75}	Average precision at IoU = 0.75
Precision	Computed using the fixed detection matching protocol
Recall	Computed using the fixed detection matching protocol
F1-score	Harmonic mean of precision and recall
Parameters	Number of model parameters
Computational cost	Measured using the selected complexity-analysis procedure
Model size	Size of the saved model checkpoint
CPU latency	Repeated inference measurements after warm-up
Throughput	Reported when measured under the same benchmark conditions

4.6 Statistical and Reproducibility Protocol

Reproducibility is maintained by fixing the dataset partition, random state, model configuration, preprocessing procedure, evaluation protocol, and benchmarking conditions.

The same held-out evaluation subset is used when comparing M0, M1, M2, M3, and M4. Calibration images used for normalization-statistics recalibration or INT8 calibration remain separated from the final evaluation data.

Whenever multiple independent training runs are performed, the corresponding results are reported using the mean and standard deviation:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

For a single experimental run, the individual measured value is reported directly rather than being presented as a multi-run statistical estimate.

CPU latency measurements follow a fixed protocol consisting of warm-up iterations followed by repeated timed inference runs under identical CPU thread settings. The reported latency is computed from the timed runs, while measurement variability is retained for analysis.

The complete experimental sequence is therefore:

$$\boxed{M0 \rightarrow M1 \rightarrow M2 \rightarrow M3 \rightarrow M4}$$

where each checkpoint is evaluated using the same core detection definitions and the same held-out evaluation protocol.

The numerical outcomes, including detection accuracy, parameter reduction, computational reduction, model-size reduction, latency, and the resulting accuracy–efficiency trade-off, are presented in Chapter 5.

Chapter 5
Results

This chapter reports what we measured on the 320-image held-out subset described in Chapter 3. AP follows the COCO protocol and is given on a scale from 0 to 1. Detectors are evaluated with the same preprocessing as in training (shortest edge 800 px, longest edge at most 1333 px). Where we compare with the reference report, we only use relative quantities, retention and relative reductions, because the two studies differ in framework, training budget, and baseline AP (Section 5.9).

We start with ResNet-18 (Section 5.1) and then move to Faster R-CNN. All Faster R-CNN students use the final recovery recipe: 1,600 iterations, training at 640–800 px, all backbone layers trainable, and a loss equal to the detection loss plus 0.5 times an MSE on the FPN levels P2–P5. The earlier recipe (800 iterations, 480–576 px, stem and `res2` frozen) is called the base recipe and appears in Section 5.5. Five pruning configurations went through $M_0 \to M_3$: internal (31.5% parameter reduction), residual (18.4%), whole backbone (41.8%), aggressive (57.6%), and maximal (61.7%). The uniform and head-protected policies were run with the final recipe at 57.6% reduction, and earlier with the base recipe at 51.8% and 48.6%. The M_4 INT8 stage was applied to the maximal configuration. Retention is $\mathrm{AP}_{\mathrm{student}}/\mathrm{AP}_{M_0}$.

5.1 ResNet-18 Study: the Pipeline on a Second Architecture

We begin with ResNet-18 because it tests a claim made in the method chapter: the ordered pipeline of structured pruning, distillation, and post-training INT8 should not depend on one detector. We ran it end to end on an ImageNet-pretrained ResNet-18, a classifier with no FPN, RPN, or RoI head. The model is registered through the framework's adapter, its channels are ranked with the framework's importance classes and global ranker, the pruning plan comes from the framework's policy, and the framework's own pruner removes the channels through its dependency graph (script `run_resnet18_framework.py`). As a control we ran the same pipeline once with Torch-Pruning called directly. Two changes were needed to move the framework from the detector to a classifier: the dependency-graph manager assumed the dictionary output of Detectron2 and now accepts a plain tensor, and the pruner batch size became configurable.

Setup. We fine-tuned an ImageNet-pretrained ResNet-18 [22] on Imagenette (10 classes, 160 px release, images resized to 128×128) for 8 epochs with Adam (learning rate 3×10^{-4}). Channels are ranked globally with first-order Taylor scores, the classifier layer is protected, and the target is 81% fewer parameters, the same as the 11.18 M→2.08 M reduction of the reference ResNet-18 study. Recovery distills the unpruned FP32 network into the pruned one (temperature $T = 4$, weight 0.3 on the cross-entropy term, Adam at 10^{-3} with cosine decay) for 15 epochs; we also report a 4-epoch run, closer to the reference budget. Quantization is static INT8 through PyTorch FX with 500 calibration images. The training subset has 8,000 images and the validation set 2,500. Every configuration was run with one seed.

Table 5.1: ResNet-18 on Imagenette, 128×128 inputs, pipeline run through the framework (15 recovery epochs). Accuracy and macro-F1 on the validation images; single run.

Stage	Acc. (%)	F1 (%)	Size (MB)	Params (M)	MACs (M)	Lat. (ms)
S0 FP32 baseline	95.00	95.02	44.80	11.18	595	14.5
S1 pruned (no recovery)	11.52	4.62	8.45	2.10	327	9.9
S2 after distillation	92.56	92.58	8.45	2.10	327	10.0
S3 INT8 (PTQ)	92.32	92.33	2.19	2.10	327	6.0

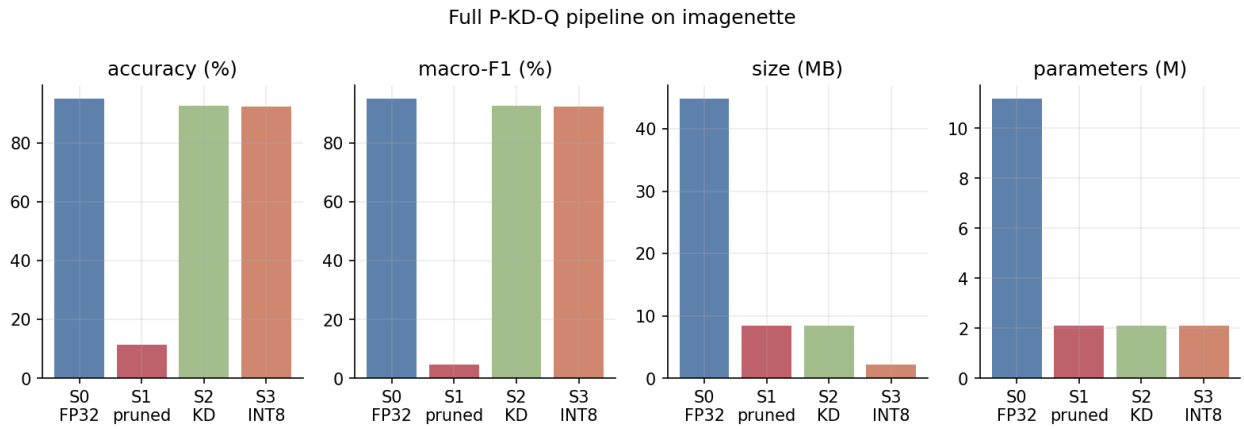


Figure 5.1: ResNet-18 pipeline on Imagenette: accuracy, macro-F1, size, and parameter count after each stage.

Findings. The framework removes 81.25% of the parameters (51 dependency groups, none rejected) and 45.0% of the multiply–accumulates (595→327 M). Without recovery the network falls to 11.5% accuracy, close to chance for ten classes, as the detector did (Section 5.4). Distillation brings it back to 92.56%, which is 97.4% of the baseline, and INT8 conversion divides the size by 3.9 (8.45→2.19 MB) for a further loss of 0.24 points. In total the model is 20.5 times smaller (44.80→2.19 MB) and loses 2.68 points of accuracy, so 97.2% of the baseline is kept. With the shorter 4-epoch recovery the chain gives 90.52% after distillation and 90.44% after INT8, so the recovery budget matters here as it does for the detector.

Framework against direct pruning. Called directly, Torch-Pruning reaches 81.06% parameter reduction, 92.20% after distillation, and 92.00% after INT8 (8.52 and 2.20 MB). The framework is 0.3 points higher at each recovered stage and prunes slightly more (81.25% against 81.06%). With one seed and 2,500 validation images, a gap this small is within sampling noise. All we can say is that the framework pruner does at least as well as the library it builds on, for this architecture.

Importance criteria. We compared four criteria at five reduction levels, each followed by 6 epochs of distillation (Table 5.2, Figure 5.2). Taylor is best from 50% to 80% reduction (93.5 and 91.1% F1, against 91.6 and 88.8 for activation, 89.6 and 87.7 for L2, and 83.8 and 82.4 for L1), and L1 is the weakest at every level up to 90%. At 20% and at 95% the criteria are close, and at 95% Taylor is the lowest of the four (77.1% F1 against 79.5 for L2), so no criterion wins everywhere. The curves of the pruned networks before recovery show the same order: at 20% reduction a Taylor-pruned network still keeps roughly two thirds of its accuracy, while L1 and L2 are already near chance.

Table 5.2: ResNet-18, macro-F1 (%) after distillation for four importance criteria and five parameter reductions; single run per cell.

Criterion	20%	50%	80%	90%	95%
L1	91.3	83.8	82.4	81.6	78.5
L2	93.9	89.6	87.7	85.9	79.5
Activation	93.4	91.6	88.8	85.5	78.2
Taylor	93.6	93.5	91.1	86.1	77.1

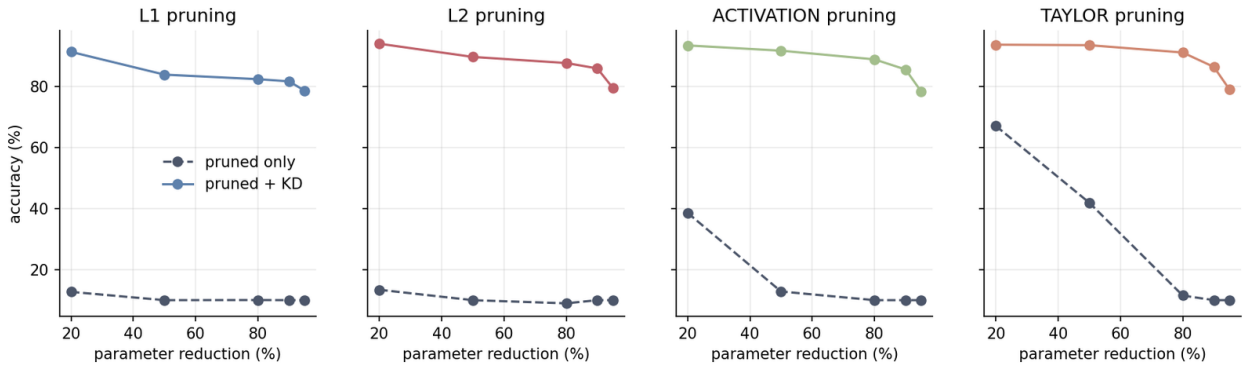


Figure 5.2: ResNet-18: accuracy of the pruned network alone and after distillation, for each importance criterion.

Distillation against cross-entropy, and re-initialisation. With Taylor ranking we compared recovery by distillation with plain cross-entropy fine-tuning, starting either from the surviving weights or from a fresh initialisation (Table 5.3, Figure 5.3). Keeping the weights is much better than starting over, by 15 to 24 points of accuracy at every level, so pruning a trained network pays off compared with training a small one from scratch. Distillation matches or slightly beats cross-entropy at 50% and 80% reduction (93.2 against 92.4, and 91.0 against 90.9), but at 90% cross-entropy is ahead by 7.5 points (88.5 against 81.0). On this classifier, distillation is not a general improvement over cross-entropy. We do not carry this result over to the detector, where distillation works on FPN feature maps and the comparison of Section 5.4 is a different one. With re-initialised weights, distillation is ahead of cross-entropy by 2.3 to 3.6 points.

Table 5.3: ResNet-18, accuracy (%) after 6 recovery epochs, Taylor ranking; single run per cell.

Reduction	CE, weights kept	KD, weights kept	CE, re-init	KD, re-init
50%	92.4	93.2	74.7	77.0
80%	90.9	91.0	72.6	76.2
90%	88.5	81.0	63.6	67.0

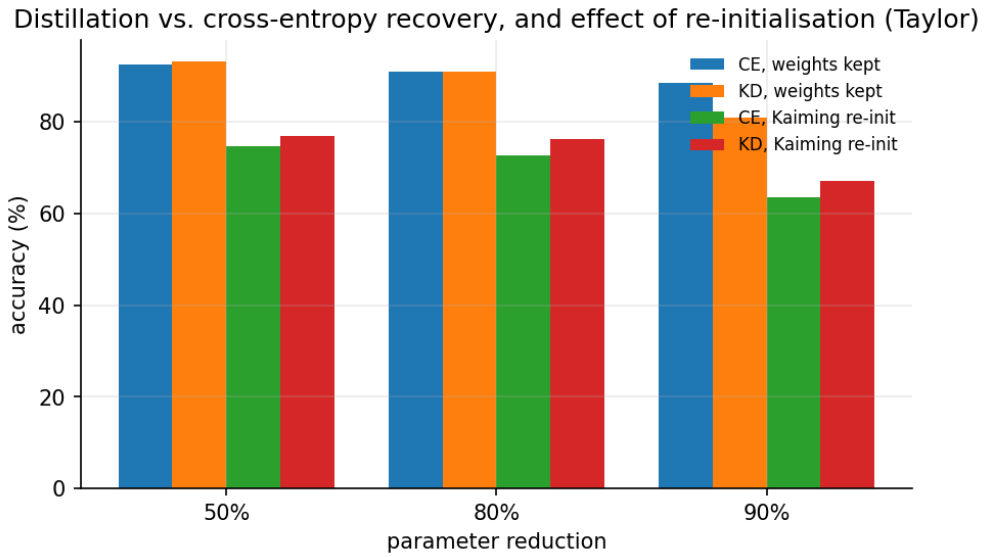


Figure 5.3: ResNet-18: distillation against cross-entropy recovery, with and without re-initialisation of the surviving weights.

Stand-alone INT8. On the unpruned network, INT8 conversion gives 94.88% accuracy against 95.00% (0.12 points lost), 11.31 MB against 44.80 MB, and 7.45 ms against 14.59 ms per image, measured back to back. Quantization alone costs almost nothing here, so the 0.24-point loss in the full chain comes from combining it with pruning and recovery.

Uncertainty. With 2,500 validation images the standard error on an accuracy near 92% is about 0.5 points, so a 95% interval is roughly ±1 point. Differences below one point (framework against direct pruning, INT8 against FP32) cannot be told apart from noise. The collapse after pruning, the recovery of more than 80 points, and the size reduction are far larger than that. Each cell was run once, and the accuracies in Tables 5.2 and 5.3 come from the shorter recovery runs and not from the 15-epoch chain.

Latency. The per-stage latencies in Table 5.1 come from one run on a shared GPU machine, and earlier runs of the same models varied a lot, so we read them as rough values. The back-to-back INT8 comparison above and the operation count (595 to 327 M MACs) are more reliable.

Scope. The study uses a classification task, a pretrained initialisation, a short recovery budget, and one dataset (Imagenette); the criteria comparison also uses Imagenette instead of the COCO subset of the reference report. Size and operation counts carry over by construction. Accuracy retention does not carry over to the detector, where only the detector experiments of this chapter count. What the study does show is that the pruner, built and tested on a detector, also prunes a classifier correctly through the same interfaces.

5.2 Baseline Faster R-CNN Inspection Performance

The reference detector M_0 is a Detectron2 Faster R-CNN R50-FPN trained for 8,000 iterations on the 1,280-image training subset. It has 41,305,311 parameters (165.6 MB in FP32). Table 5.4 shows that it solves the task before any compression.

Table 5.4: Baseline detector M_0 on the 320-image held-out subset.

Metric	Value
$AP_{50:95}$	0.7119
AP_{50}	0.9215
AP_{75}	0.8551
Precision	0.8228
Recall	0.9426
F1-score	0.8786
Jaccard	0.7836
TP / FP / FN	3140 / 676 / 191
Number of parameters	41,305,311
Model size (FP32)	165.6 MB
Inference latency	8126 ms

AP_{50} is above 0.92, so insulators are located reliably at the permissive IoU threshold, and the AP_{75} of 0.855 shows that the boxes are also tight. Per-class AP is 0.769 for *No issues*, 0.631 for *Broken*, and 0.735 for *Flashover damage*. *Broken* is the weakest class and also the least represented (1,180 annotations, Table 3.1). Precision, recall, and F1 are computed at score threshold 0.5 and IoU 0.5 for all models. Cost is measured with a

fixed protocol: GMACs counted with fvcore (FLOPs= 2×MACs), and latency as the median of 20 single-image runs after 3 warm-up runs (batch size 1, 4 CPU threads, first test image of 800 × 1200 px, Intel Core i5-6200U). The baseline costs 192.3 GMACs (384.6 GFLOPs). The confusion matrix, per-class precision and recall, and IoU distribution of M_0 are in Appendix C.

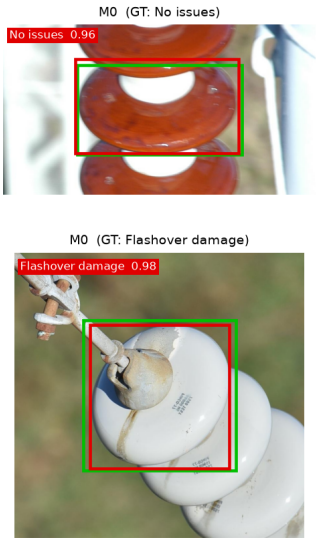


Figure 5.4: Qualitative baseline detections (M_0) on easy, difficult, distant, and damaged insulators. Ground truth in green, prediction in red.

5.3 Faster R-CNN Component Sensitivity Ablation

All configurations use channel-level Taylor importance, $I_c = |W_c \, \partial L / \partial W_c|$ [25], with rank-normalized global ranking (Chapter 3). Only the way the pruning budget is shared out changes between policies, so this ablation asks whether it matters where capacity is removed. Table 5.5 gives all policies after the full recovery pipeline (M_3), all with the final recipe. We first ran the uniform and head-protected policies with the base recipe (800 iterations, other reduction levels, Section 5.5) and then repeated them with the final recipe at the reduction of the aggressive configuration (57.6%), so the three policies at 57.6% can be compared directly.

Table 5.5: Pruning-policy ablation after recalibration and distillation (M_3, FP32). All rows use the final recipe (1,600 iterations). The three rows at 57.6% share the same reduction target. Parameters of the internal and residual rows are derived from their reduction.

Policy	**Red.**	Params	$AP_{50:95}$	AP_{50}	AP_{75}	F1	Ret.
Teacher M_0	0%	41.31 M	0.7119	0.9215	0.8551	0.879	100%
Teacher M_0 fine-tuned (final recipe, M0 control)	0%	41.31 M	0.7528	0.9422	0.8824	0.896	105.7%
Residual-stream pruning	18.4%	33.71 M	0.7473	0.9412	0.8863	0.897	105.0%
Internal pruning	31.5%	28.30 M	0.7403	0.9330	0.8735	0.890	104.0%
Whole backbone, critical modules protected	41.8%	24.03 M	0.7199	0.9183	0.8502	0.877	101.1%
Aggressive backbone, critical modules protected	57.6%	17.51 M	0.6079	0.8056	0.7248	0.823	85.4%
Head protected (matched 57.6%)	57.6%	17.52 M	0.5901	0.7861	0.7025	0.808	82.9%
Uniform, rank-normalized (matched 57.6%)	57.6%	17.52 M	0.7032	0.9001	0.8298	0.870	98.8%
Maximal pruning	61.7%	15.81 M	0.5751	0.7692	0.6785	0.816	80.8%

Retention falls as more capacity is removed once the reduction passes about 42% for the protected policies: 101.1% at 41.8%, 85.4% at 57.6%, and 80.8% at 61.7%.

Three configurations (18.4%, 31.5%, and 41.8% reduction) keep more than 100% of the teacher AP, which means the student beats the teacher on this subset. The recovery stage is extra training at the evaluation resolution, which the teacher never received. To check this we fine-tuned the unpruned M_0 with the same recipe

(M0 control), and it reached AP = 0.7528 (105.7%). Against this fine-tuned teacher the students keep 99.3% (residual), 98.3% (internal), 95.6% (whole backbone), 80.8% (aggressive), and 76.4% (maximal). The values above 100% therefore come from the extra training and not from pruning.

At a matched 57.6% reduction and with the same recipe, the three allocations came out in the opposite order from what we expected. The rank-normalized uniform policy keeps 98.8% (93.4% of the fine-tuned teacher), the policy that protects the stem, RPN, and heads keeps 85.4%, and the head-protected policy keeps 82.9%. We think the reason is where the parameters are removed. The uniform policy prunes about 45% of the channels of every group, including the fully connected RoI layers, whereas the head-protected policy has to prune about 61% of the backbone channels to reach the same total. The RoI head holds a large share of the parameters of a three-class detector and seems to be more redundant than the backbone. The numbers fit this explanation, but no separate experiment isolates it.

Protecting the prediction modules therefore did not give better compression at a given parameter count in our data. Retention does depend on how the budget is shared: 15.9 points separate the uniform and head-protected policies at the same reduction. Each cell is one run on 320 images.

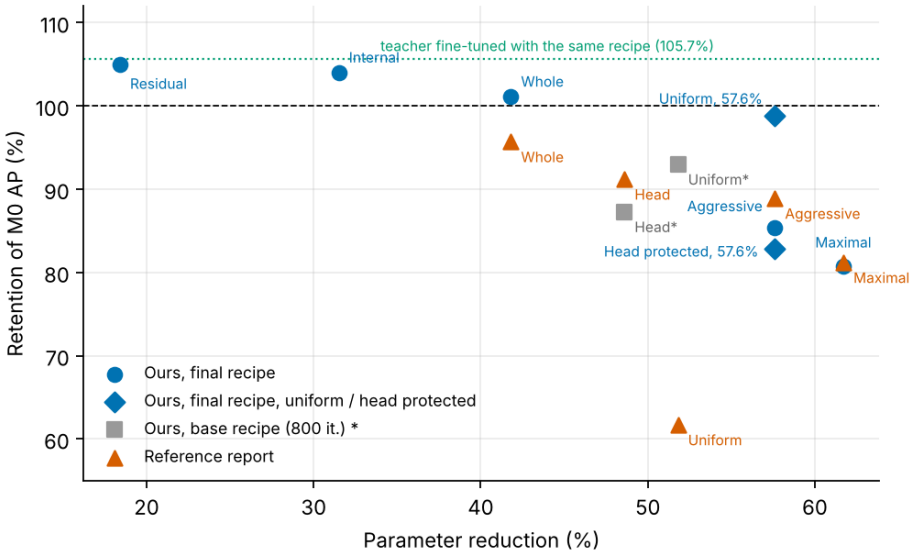


Figure 5.5: Retention versus parameter reduction. Circles: this study with the final recipe; squares (*): base recipe, 800 iterations; triangles: reference report.

5.4 Effect of Recalibration and Knowledge Distillation

Table 5.6 reports the same student at the four checkpoints, which isolates the contribution of each recovery step.

Table 5.6: $AP_{50:95}$ (retention) of the student at each stage (M_3: final recipe, 1,600 iterations).

Configuration	M_0	M_1 pruned	M_2 recalibrated	M_3 distilled
Whole backbone (41.8%)	0.7119	0.0047 (0.7%)	0.279 (39.2%)	0.7199 (101.1%)
Aggressive (57.6%)	0.7119	0.0000 (0%)	0.0177 (2.5%)	0.6079 (85.4%)
Maximal (61.7%)	0.7119	0.0000 (0%)	0.0037 (0.5%)	0.5751 (80.8%)

Right after pruning (M_1), the whole-backbone student has lost essentially all detection ability (0.7% retention). Removing channels with no adaptation breaks the match between the learned weights and the normalization statistics. Recalibration (M_2) restores part of the performance when pruning is moderate (+38.5 retention points for the whole backbone) but almost nothing under stronger pruning (2.5% and 0.5%), which is expected from a step that updates statistics and no weights. For the aggressive and maximal configurations we ran recalibration block by block in sequence, because a simultaneous update did not converge. Distillation (M_3) gives most of the recovery: +44.1, +59.0, and +57.1 AP points for the three configurations. For the aggressive

and maximal students M_1 outputs no detection at all (AP= 0), so retention is 0%.

The gain from M_2 to M_3 mixes supervised detection fine-tuning and feature distillation. To separate them we trained the aggressive student with the task loss only ($\beta = 0$, same recipe, same seed): AP = 0.5908 (retention 83.0%, F1 0.816), against 0.6079 (85.4%, F1 0.823) with $\beta = 0.5$. The FPN feature term therefore adds 1.7 AP points, or 2.4 retention points. Three seeds of the $\beta = 0.5$ aggressive student give AP 0.6079, 0.6154, and 0.6070 (mean 0.6101, standard deviation 0.0046, retention $85.7 \pm 0.6\%$). The $\beta = 0$ run is below all three, but it is a single run, so the effect is consistent without being statistically established. Our first distillation runs were also short (800 iterations at 480–576 px, stem and `res2` frozen) so that they would fit on CPU-only hardware; Section 5.5 measures what a longer recipe changes.

Distillation budget. To see whether the weaker recovery of strongly pruned students comes from the budget, we distilled the aggressive student a second time for 1,200 iterations instead of 800 (same recipe and seed).

Table 5.7: Aggressive student (57.6% reduction) for two distillation budgets.

Budget	$AP_{50:95}$	AP_{50}	AP_{75}	P	R	F1	Ret.
800 iterations	0.4877	0.6796	0.5894	0.713	0.825	0.765	68.5%
1,200 iterations	0.5158	0.7109	0.6113	0.723	0.838	0.776	72.5%

A 50% longer distillation (about 5.1 h instead of roughly 3.4 h on the CPU) improves AP by 0.028, or 4.0 retention points, with gains on every metric. We only tried two budgets and draw no trend from them.

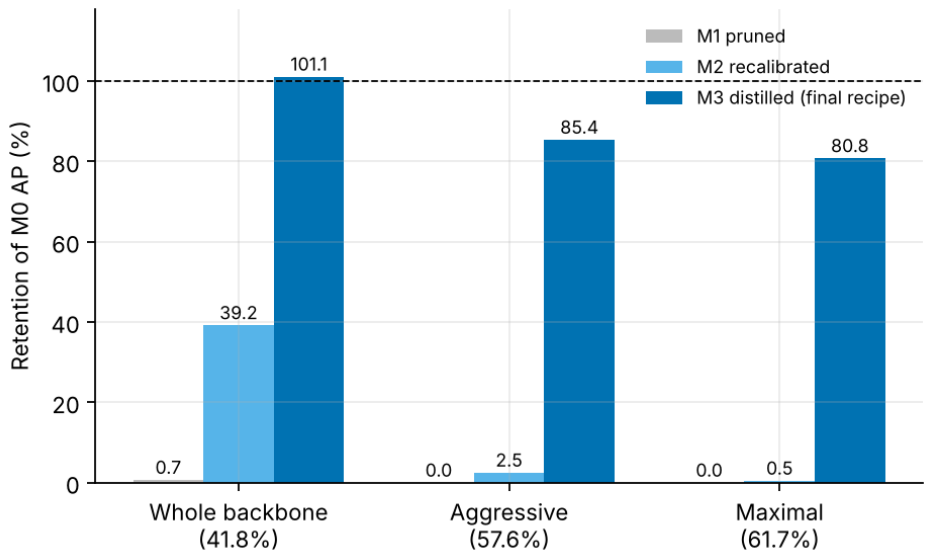


Figure 5.6: Retention at each stage (M_1 to M_3, final recipe) for the three main configurations.



Figure 5.7: Distillation training curves (final recipe, 1,600 iterations; moving average over 7 logged points): total loss, task loss, and FPN feature-alignment loss.

5.5 Recovery Recipe, Uniform Allocation and Zero-Out Test

The gap in Table 5.12 for strong pruning led us to study the recovery recipe on the aggressive student (same M_2, same seed). We tested three changes alone and together: multi-scale training at 640–800 px (maximum 1333) instead of 480–576 px, the resolution used at evaluation; unfreezing the stem and `res2` (`freeze_at`=0); and an extra class-logit distillation term on the RoI head (KL divergence, $T = 2$, weight 1.0). The class-logit term is our own addition and is not in the reference loss. Combining the three changes with 1,600 iterations gives the combined variant. The final recipe is the same combination without the class-logit term, and it produces the headline numbers.

Table 5.8: Recovery-recipe ablation on the aggressive student (57.6% reduction); one seed. Retention is relative to M_0.

Variant	**Change**	$AP_{50:95}$	**Ret.**
Base (800 it.)	480–576 px, stem+res2 frozen, $\beta = 0.5$	0.4911	69.0%
Resolution	+ evaluation-scale training resolution	0.5374	75.5%
Unfrozen layers	+ stem and res2 trainable	0.5134	72.1%
Class-logit term	+ class-logit KD (extension)	0.5037	70.7%
Combined	all three, 1,600 it.	0.6249	87.8%
Final recipe	all three changes without class-logit KD	0.6079	85.4%
final recipe, seeds 1 and 2	same recipe, other seeds	0.6154 / 0.6070	86.4% / 85.3%
final recipe, $\beta = 0$	final recipe without the FPN feature term	0.5908	83.0%

The single changes add +6.5, +3.1, and +1.8 retention points, 11.4 in total, whereas the combination with 1,600 iterations adds +18.8. The longer budget cannot be separated from the other changes. The base row of this table (0.4911) differs by 0.003 AP from the base run of Table 5.7 (0.4877), which is the size of the run-to-run variation. The class-logit term adds 1.7 AP points over the final recipe on the aggressive student, +0.9 on the maximal student (0.5843 against 0.5751), and −0.2 on the whole-backbone student (0.7176 against 0.7199). It is not needed, and we left it out of the headline results.

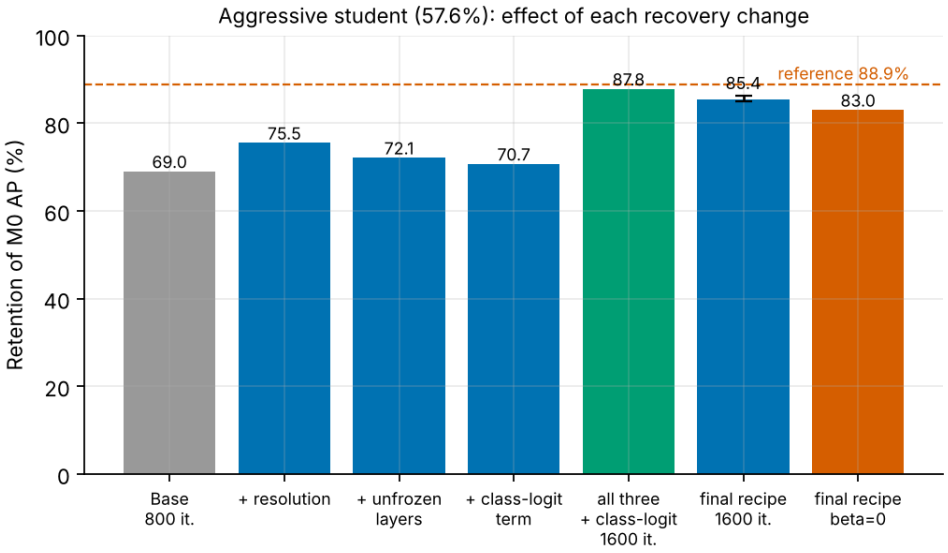


Figure 5.8: Retention of the aggressive student for each recovery change. The error bar is the standard deviation over three seeds of final recipe; the dashed line is the reference value.

Uniform allocation. Our uniform policy ranks channels with rank-normalized scores and equal group weights, so every group is pruned by about the same share and the fully connected RoI layers are not emptied (the reference reports −99% and −98% for fc6 and fc7). With the base recipe it kept 93.0% at 51.8% reduction, while the reference uniform pruning kept 61.7%, so we did not reproduce the reference uniform baseline. The base-recipe head-protected policy kept 87.3% (reference 91.2%). We repeated both policies with the final recipe at 57.6% reduction (Section 5.3): uniform keeps 98.8% and head-protected 82.9%. The uniform allocation of our implementation does not collapse the way the reference one did.

Zero-out test. The reference pipeline may zero filters and then remove them. We implemented and tested this. It gave a block-wise deviation of 12–14.6 points and a reduction error of at least 11.5 points, against 6.43 points for channel ranking with rank clipping, and it did not reproduce the reference maps. We kept channel ranking and declare the difference in Appendix E.

5.6 INT8 Quantization Results

We applied post-training INT8 quantization per unit (stem, `res2`–`res5`, RoI box head) with BatchNorm folding and static FX conversion. The FPN, RPN, and final predictor stay in FP32. Calibration used 200 training images. On the laptop the backend is `onednn`, the only one exposed by the installed PyTorch build; the repeat described below ran on a cloud CPU with `fbgemm`. Any unit can be kept in FP32, which lets us measure the sensitivity of each one.

Table 5.9: INT8 results on the maximal student. Retention is relative to M_0.

Model	Params (M)	Size (MB)	$AP_{50:95}$	**Ret.**
FP32 teacher M_0	41.31	165.6	0.7119	100%
Maximal, FP32 (M_3, final recipe)	15.81	63.3	0.5751	80.8%
Stem only in INT8 (other units FP32)	15.81	–	0.4289	60.2%
Final M_4: all units INT8 except the stem	15.81	18.2	0.5712	80.2%
Earlier M_4: all units INT8 on the base-recipe M_3	15.81	18.2	0.4249	59.7%

Quantizing the stem alone lowers retention from 80.8% to 60.2%, a drop of 20.6 points, which is about the whole loss of the earlier all-units INT8 model (0.4638 to 0.4249 on its own M_3). The stem is the first convolution applied to the raw image, and it is the unit of this detector most sensitive to quantization, so the final M_4

keeps it in FP32. This differs from the reference configuration, which quantizes the stem too (Appendix E). The earlier all-units model cut the serialized size from 63.3 to 18.2 MB (3.5×, or 9.1× relative to M_0); the RoI box head holds most of the parameters. The final model reaches AP = 0.5712 (AP_{50} 0.765, AP_{75} 0.673), which is 80.2% of the teacher and 99.3% of its FP32 M_3, at 18.2 MB (3.5× smaller than the FP32 student and 9.1× smaller than M_0). Keeping the stem in FP32 removes almost all of the quantization loss (0.5 retention points).

Second training and F1 of the INT8 model. We had not recorded the F1 of the final INT8 model in the first run, so we trained the maximal student a second time with the same recipe and evaluated its INT8 version (stem in FP32) on a cloud CPU. The second FP32 student reaches AP = 0.5770 (81.1%, F1 = 0.815), against 0.5751 for the first, which gives an idea of the run-to-run variation of the recipe (0.2 AP points). Its INT8 version reaches AP = 0.5738 (AP_{50} 0.765, AP_{75} 0.684), a retention of 80.6%, with precision 0.762, recall 0.867, and F1 = 0.811, which is 0.003 below its FP32 version, at 18.2 MB. With the stem in FP32, quantization costs 0.5 to 0.6 points.

Latency. The earlier all-units model was 26% slower than its FP32 counterpart on the evaluation CPU (3983 against 3155 ms, same session). The FPN, RPN, and predictor stay unquantized, which limits the part of the network that can speed up, and a gain should be checked on hardware with native INT8 kernels. Absolute latencies on this laptop vary a lot between sessions (the maximal FP32 model took 3155 ms in the INT8 session and 4786 ms, with a standard deviation of about 1000 ms, in the cost session of Table 5.10), so we only interpret ratios measured within one session. For the final model the median latency is 4477 ms against 5139 ms for its FP32 counterpart in the same session, a 1.15× speed-up. With the stem in FP32 the INT8 model is faster, unlike the earlier all-units model. On the cloud CPU (4 threads) the INT8 model was faster in three separate sessions: 2130, 2176, and 2077 ms in FP32 against 1594, 1627, and 1639 ms in INT8, speed-ups of 1.34×, 1.34×, and 1.27×. The gain depends on the machine and backend (1.15× on the laptop in one session, 1.27 to 1.34× on the cloud CPU). We made no measurement on embedded hardware.

5.7 End-to-End Accuracy–Efficiency Trade-Off

Table 5.10 summarizes all checkpoints. The whole-backbone model is the best moderate operating point (best retention), the maximal model is the most efficient, and the INT8 model is the smallest.

Table 5.10: Accuracy–efficiency summary (cost session: 800 × 1200 input, median of 20 runs, 4 threads, i5-6200U).

Model	$AP_{50:95}$	Ret.	Size (MB)	GFLOPs	ΔGFLOPs	Latency (ms)	Speed-up
Baseline M_0	0.7119	100%	165.6	384.6	–	8126	1.0×
Preferred moderate (whole)	0.7199	101.1%	96.4	268.6	−30.1%	5842	1.4×
Aggressive	0.6079	85.4%	70.2	222.7	−42.1%	5402	1.5×
Maximal (FP32)	0.5751	80.8%	63.3	211.2	−45.1%	4786	1.7×
Maximal + INT8 (final)	0.5712	80.2%	18.2	partial	–	–	1.15× vs its FP32

On disk, the three FP32 students are 1.7×, 2.4×, and 2.6× smaller than M_0. The final INT8 model is 9.1× smaller and runs 1.15× to 1.34× faster than its FP32 counterpart in the same session, depending on the machine (Section 5.6). The saving in computation (30–45% of GFLOPs) is smaller than the reduction in parameters (42–62%), because the pruned channels sit mostly in the deeper stages, which hold many parameters but work on small feature maps. Accuracy and size come from the final recipe, and the cost columns come from the architecture, which the recipe does not change. In Pareto terms, the whole-backbone model keeps the teacher AP with 30% fewer GFLOPs. Going on to the maximal model saves another 33 MB and about 1.1 s per image but costs 20.3 retention points, and INT8 is worth it when storage is the binding constraint.

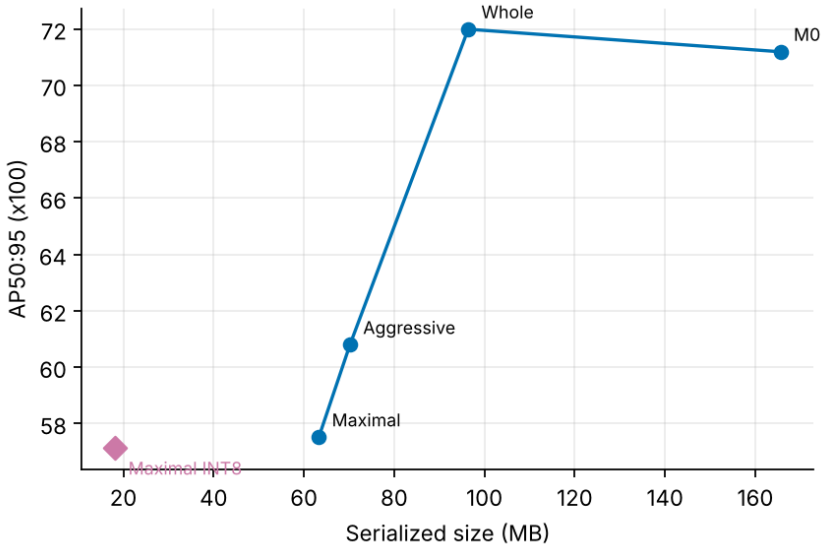


Figure 5.9: $AP_{50:95}$ versus model size.

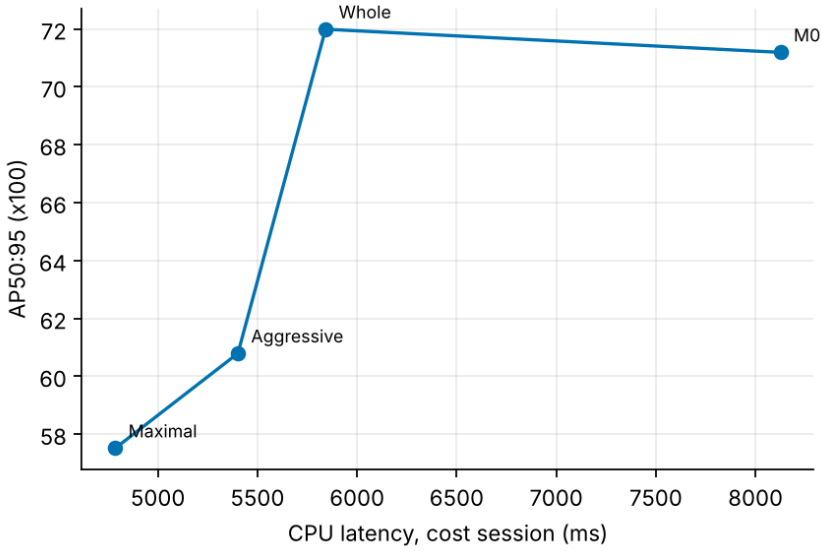


Figure 5.10: $AP_{50:95}$ versus CPU latency (cost session). The INT8 model is not shown because it was timed in a different session.

5.8 Error Analysis

Figures 5.11 and 5.12 show the same insulator for M_0, M_1, M_2, and M_3 (ground truth in green, prediction in red, cropped around one insulator), with the same preprocessing as the quantitative evaluation. They agree with the numbers: M_1 usually outputs no detection, M_2 gives sparse or poorly placed boxes, and for the moderate configuration M_3 recovers detections close to those of M_0.

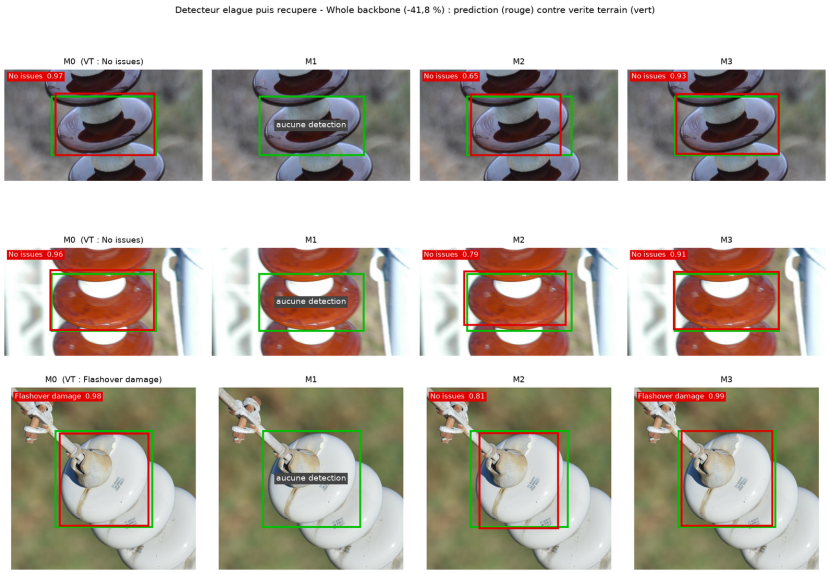


Figure 5.11: Whole backbone: one insulator at M_0, M_1, M_2, M_3.

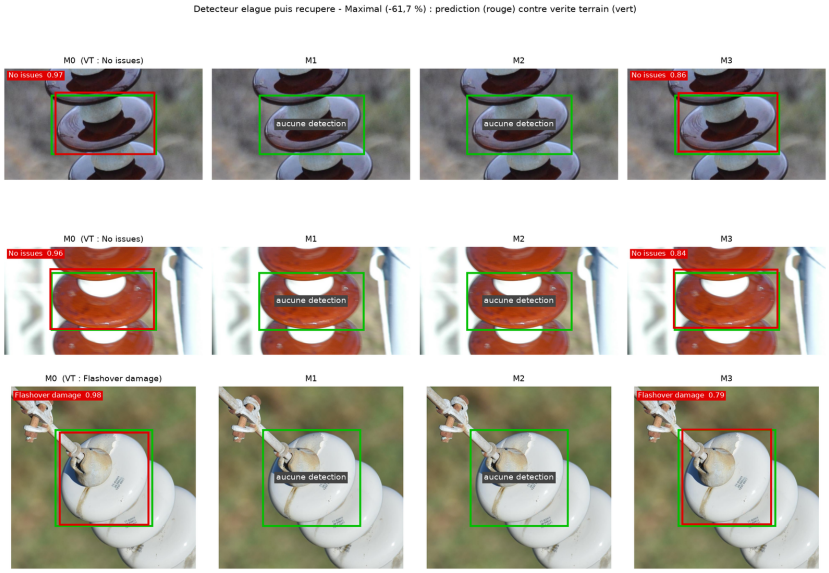


Figure 5.12: Maximal pruning: one insulator at M_0, M_1, M_2, M_3.

The metrics already hint at the type of error: AP_{75} retention is lower than AP_{50} retention for every student (maximal model: 79.4% against 83.5%), so box placement degrades more than detection itself.

We then classified the errors from the detections themselves, on the 320 test images, at score 0.5 and IoU 0.5, for M_0, a second training of the maximal student with the same recipe (M_3), and its INT8 version (M_4). Each missed object gets one cause: class confusion (a detection of another class overlaps it with IoU ≥ 0.5 and better than any detection of its own class), poor localization (a detection of the right class overlaps it with IoU between 0.1 and 0.5), lower rank or duplicate (a matching detection exists but was used for another object), or missed entirely. False positives are classified the same way, with background when nothing overlaps them. Size classes are tertiles of the ground-truth box area in the test set, and scene classes count the annotated objects per image. Table 5.11 and Figure 5.13 give the results.

Table 5.11: Error analysis on the 320 test images (score ≥ 0.5, IoU ≥ 0.5). Recall by condition class, size tertile and scene density, and counts of missed objects (FN) and false detections (FP) by cause. M_3 and M_4 are the second training of the maximal student and its INT8 version.

	M_0	maximal M_3	maximal M_4 (INT8)
Recall, No issues (2,565 objects)	96.8%	94.8%	94.7%
Recall, Broken (243)	89.3%	59.7%	60.1%
Recall, Flashover damage (523)	83.9%	61.2%	60.2%
Recall, small / medium / large objects	93.9 / 94.2 / 94.8%	85.2 / 87.9 / 87.7%	84.6 / 88.0 / 87.6%
Recall, images with 2–3 / 4 or more objects	94.9 / 94.3%	82.1 / 87.0%	82.1 / 86.8%
Missed objects (FN), total	191	435	442
class confusion	131	288	296
poor localization	17	48	47
lower rank or duplicate	29	24	25
missed entirely	14	75	74
False detections (FP), total	676	882	902
class confusion	291	465	477
poor localization	203	180	191
background	167	218	220
duplicate	15	19	14

The loss caused by compression sits on the two damage classes. Recall falls from 89.3% to 59.7% for broken insulators and from 83.9% to 61.2% for flashover damage, while the healthy class loses only two points. Most missed objects are confused with another class and not missed entirely: 288 of 435 for the maximal student (66%) and 131 of 191 for M_0 (69%). The compressed network still finds the insulator but loses the difference between healthy and damaged. Objects missed entirely do increase, from 14 to 75. Object size has a moderate effect (recall from 94% down to about 85–88%), and the INT8 model reproduces the profile of its FP32 version almost exactly. For inspection this matters, because the average AP hides a larger loss on exactly the classes the inspection is meant to find. The analysis rests on one run per model, and the three size classes are relative to this test set. Recall for scenes with a single object is not reported because only three such objects exist. The next subsection looks at the failure images themselves.

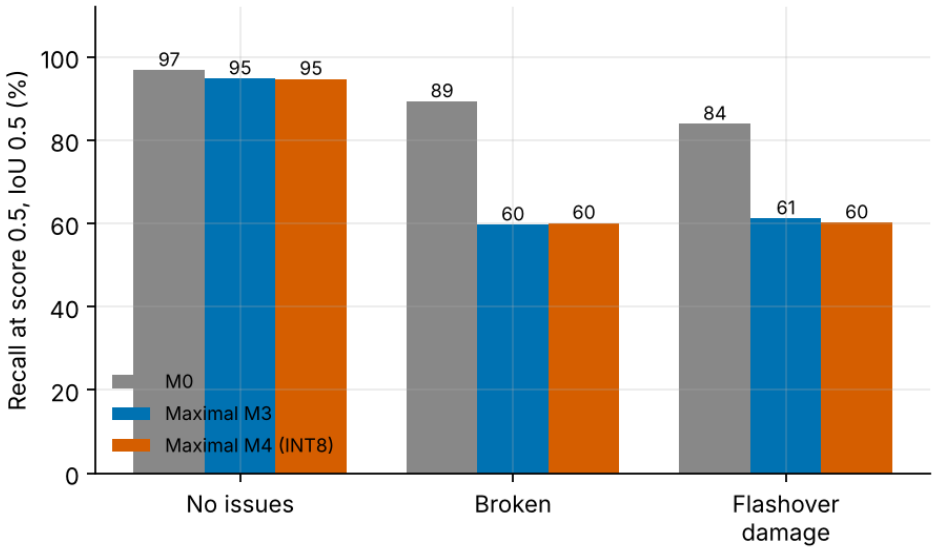


Figure 5.13: Recall per condition class at score 0.5 and IoU 0.5: the damage classes carry the loss of the compressed students.

5.8.1 Failure Images

To see what these errors look like, we listed every object that M_0 detects correctly (score ≥ 0.5, IoU ≥ 0.5, right class) and that the maximal student does not, using the first training of the maximal student (AP 0.5751). We chose the maximal student because it is the most compressed model (61.7% fewer parameters) and the final INT8 model is built from it, so its errors are the ones that matter for deployment; a less pruned student makes fewer of these errors. A case counts as a failure when the output disagrees with the ground truth, either because the class is wrong or because the object is not detected, even if the box is well placed. There are 295 such objects among the 320 test images: 206 are confused with another class, 48 are poorly localized, and 41 are missed entirely. We then drew, for each cause, six cases chosen at regular intervals after sorting by class and image, with no selection by appearance. Figure 5.14 shows four of them, with M_0 on the left and the student on the right (ground truth in green, predictions in red, crops around the object).

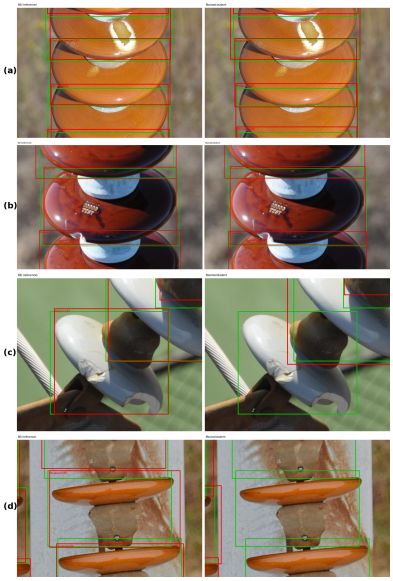


Figure 5.14: Failure cases of the maximal student. Left: M_0; right: student. (a) A flashover-damaged insulator that M_0 labels *Flashover damage* and the student labels *No issues*. (b) A broken insulator that M_0 labels *Broken* (0.93) and the student labels *No issues* (0.61) on almost the same box. (c) A broken insulator with a chipped cap that the student does not detect. (d) Healthy insulator units in a busy scene where the student outputs no box.

In the confusion cases (a) and (b) the box is well placed, and the failure is the class: a damaged insulator is reported as healthy, which is the costly error in an inspection. In (c) and (d) there is no red box at all, so the object is not detected. In the confusion cases the boxes of the student are almost the same as those of M_0, and only the label changes, often with a lower score (for example 0.61 against 0.93 in panel (b)). The damage is visible to the eye in these images: a burned area in (a), a broken edge in (c). The student seems to have lost the fine texture cues that separate a damaged unit from a healthy one, which agrees with the drop of recall on the two damage classes. The missed healthy units, as in (d), come from crowded scenes with overlapping discs or from units cut by the image border. The poor-localization cases are mostly dense strings of identical discs, where several boxes overlap and a shift of a few pixels changes which box matches which object. We looked at 18 cases in total, and this description is a reading of them, not a measurement. The gallery uses the first training of the maximal student, while Table 5.11 uses the second, so the counts differ slightly.

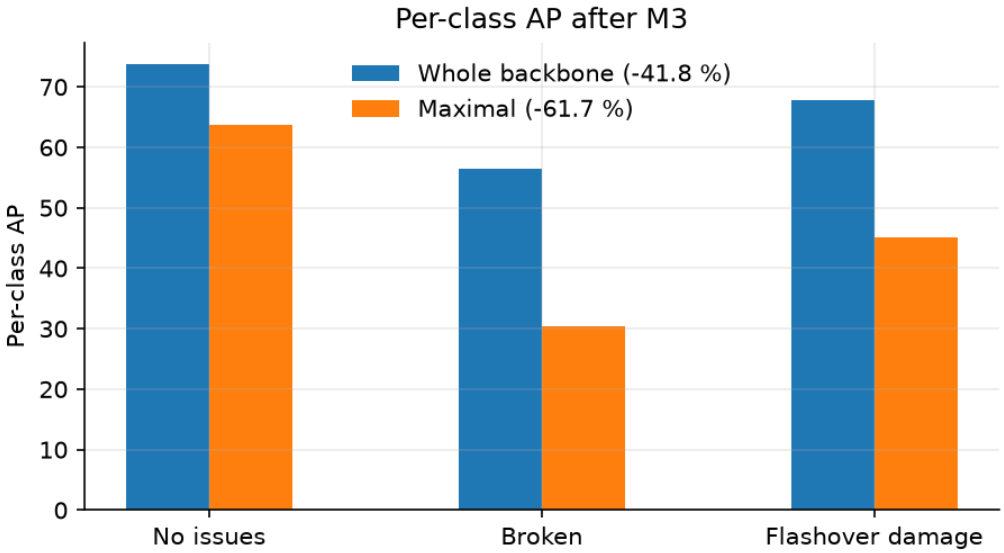


Figure 5.15: AP per condition class for M_0 and the compressed students.

5.9 Comparison with the Reference Experimental Protocol

The reference report applies the same ordered pipeline with a torchvision Faster R-CNN, a 5,000-iteration baseline ($AP_{50:95} = 0.8314$), and its own recovery budget. We use Detectron2, an 8,000-iteration baseline ($AP_{50:95} = 0.7119$), channel-level Taylor importance with rank normalization, and a short distillation limited by CPU hardware. Absolute APs cannot be compared, so Table 5.12 compares retention.

Table 5.12: Retention ($AP_{50:95}$ relative to the respective teacher): reference report [27] versus this study.

Policy	**Red.**	Reference	Ours	Gap (pts)
Uniform pruning (base recipe)	51.8%	61.7%	93.0%	+31.3
Uniform pruning (final recipe, 57.6%)	51.8%	61.7%	98.8%	+37.1
Head protected (base recipe)	48.6%	91.2%	87.3%	−3.9
Head protected (final recipe, 57.6%)	48.6%	91.2%	82.9%	−8.3
Whole backbone (800 it.)	41.8%	95.7%	92.7%	−3.0
Whole backbone (final recipe)	41.8%	95.7%	101.1%	+5.4
Aggressive (800 it.)	57.6%	88.9%	68.5%	−20.4
Aggressive (1,200 it.)	57.6%	88.9%	72.5%	−16.4
Aggressive (combined, with class-logit term)	57.6%	88.9%	87.8%	−1.1
Aggressive (final recipe)	57.6%	88.9%	85.4%	−3.5
Maximal (800 it.)	61.7%	81.2%	65.2%	−16.0
Maximal (final recipe)	61.7%	81.2%	80.8%	−0.4
Maximal + INT8 (earlier, all units)	61.7%	80.5%	59.7%	−20.8
Maximal + INT8 (final, stem FP32)	61.7%	80.5%	80.2%	−0.3

With the final recipe the retentions are within about four points of the reference for the aggressive (85.4% against 88.9%) and maximal (80.8% against 81.2%) students, and above it for the whole-backbone student (101.1% against 95.7%). That last value exceeds 100% because the teacher itself gains 4.1 AP points from the same recipe; against that fine-tuned teacher the student keeps 95.6%. With the earlier 800-iteration recipe the gap was 16 to 20 points, so most of it came from the recovery recipe (resolution, unfrozen backbone, budget) and not from pruning. The uniform and head-protected rows of the final recipe were run at 57.6% reduction, while the reference values correspond to 51.8% and 48.6%, so their gaps mix the recipe effect and the reduction level and should be read loosely. We used one seed and 320 test images, so we do not interpret differences of one or two points.

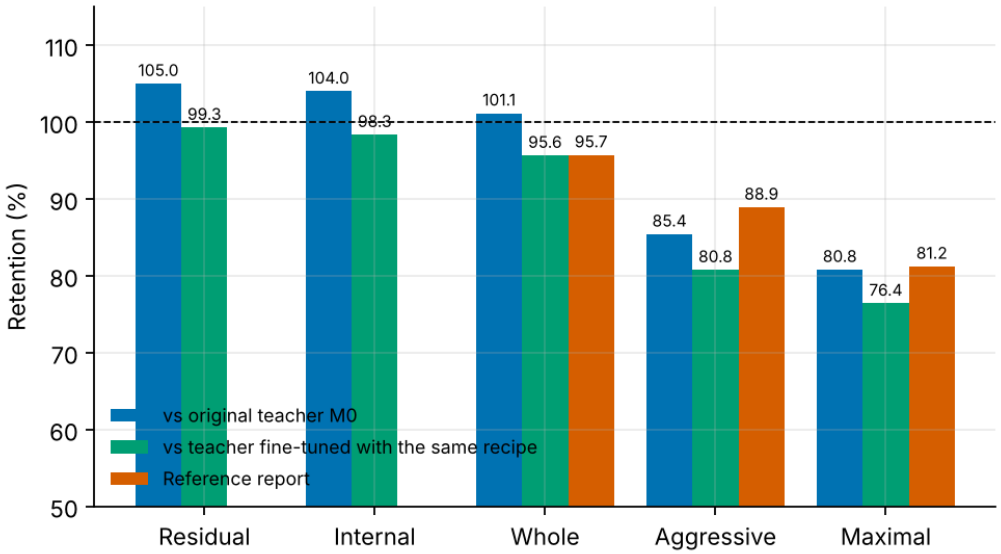


Figure 5.16: Retention of every student of the final recipe computed against the original teacher M_0 and against the teacher fine-tuned with the same recipe, next to the reference report.

Chapter 6
Discussion

6.1 Main Scientific Findings

Structural pruning alone destroys the detector: a 41.8% channel reduction drops retention to 0.7%, so a recovery stage is always needed. Most of the recovery comes from distillation-based retraining. Recalibration helps only when pruning is mild, and the FPN feature term adds a smaller part (aggressive student: 85.4% with it, 83.0% without, one seed). The recovery recipe also matters a lot: the same aggressive student goes from 69.0% retention with the base recipe to 85.4% with training resolution, unfrozen backbone, and a longer budget, so a retention figure only makes sense together with its recipe. The unpruned teacher gains 4.1 AP points from the same recipe, which explains retentions above 100%. With the final recipe the differences to the reference report are +5.4 points (whole backbone), −3.5 (aggressive), and −0.4 (maximal).

6.2 Why Architecture-Aware Protection Matters

The RPN and RoI head produce the proposals and the final class and box outputs, so an error there reaches every detection, while redundancy in the ResNet backbone can be absorbed by later stages. The whole-backbone policy follows this idea (stem, RPN, and heads protected) and keeps 95.6% of the fine-tuned teacher AP. Our own ablation does not show that protection gives better compression, though. At a matched 57.6% reduction with the final recipe, the uniform policy keeps 98.8% of the teacher AP, against 85.4% for the protected whole-backbone policy and 82.9% for the head-protected policy. The uniform policy spreads the removal over the whole network, including the RoI fully connected layers, which hold many parameters and look redundant for three classes. The protected policies have to take the same parameter budget from the backbone. The reference report argues that nominal sparsity is not useful compression, and our data add a detail: what matters is which parameters are removed, and in this detector the RoI head seems a better place to remove them than the backbone. These are single runs on one dataset.

6.3 Role of Knowledge Distillation

We see distillation as a way to recover from structural damage. The pruned student keeps the layer layout of the teacher, so matching FPN features is natural, and the gains (up to +59 AP points over M_2) show that retraining is essential. The feature term itself adds 1.7 AP points (2.4 retention points) over fine-tuning alone on the aggressive student, with one seed.

6.4 Contribution of Quantization

After recovery, quantization cut the storage of the maximal model by 3.5×. Quantizing the stem causes most of the accuracy loss (20.6 retention points when quantized alone). With the stem kept in FP32 the INT8 student loses only 0.5 retention points (80.2% against 80.8%) and runs 1.15× faster than its FP32 counterpart on the evaluation CPU, in a single session. Pruning, then distilling, then quantizing makes sense for storage, and for speed once the stem is excluded.

6.5 Implications for UAV Edge Deployment

On the 2.3 GHz dual-core laptop CPU the compressed models need between 4.8 and 5.8 s per image, far from real time, so our results do not support a real-time onboard claim. The models are smaller, cheaper, and faster than the teacher (1.7× at best in FP32), which is a step toward onboard execution, but this still has to be checked on the target hardware.

6.6 Comparison with the State of the Art

UAV-edge frameworks in the literature use different detectors, datasets, and accelerators [19], and compression studies differ in baseline and metric. We therefore give no table of direct comparison, which could mislead. The only comparison we report is the controlled one with the reference protocol (Table 5.12), in retention.

6.7 Toward Embedded Hardware

We have not run anything on embedded hardware. Appendix D lists candidate platforms, looks at the Raspberry Pi 5, and gives the steps and measurements needed to bring the framework to a board.

6.8 Limitations and Threats to Validity

- Single detector, single dataset, single split. Seed variation was measured only for the aggressive student of the final recipe (standard deviation 0.46 AP points), and we have not tested other sites or other detectors (one-stage, transformer).
- Partly open ablation. The $\beta = 0$ control and the recipe ablation were run on the aggressive student only, the importance criteria were compared on ResNet-18 only, and the uniform and head-protected policies were run at one reduction level (57.6%) with one seed.
- Retention above 100%. For the whole-backbone, internal, and residual configurations it comes from extra training: the unpruned teacher fine-tuned with the same recipe reaches 105.7%. Section 5.3 gives retentions relative to that fine-tuned teacher.
- Partial INT8. FPN, RPN, and predictor stay in FP32, so the reported FLOPs do not equal integer operations, and INT8 latency depends on the backend (here `onednn`).
- Latency on one platform. We timed a single i5-6200U CPU with 4 threads. Energy, peak RAM, temperature, and battery impact were not measured.
- Different frameworks. We use Detectron2 and the reference uses torchvision, so only retention can be compared.
- Implementation. Channel removal uses our own dependency graph, checked on this architecture only. Distillation used the 1,276 training images that contain at least one annotation.

Chapter 7
Conclusion and Future Work

7.1 Conclusion

We asked whether a high-capacity Faster R-CNN detector can be compressed for UAV power-line inspection with an ordered pipeline of pruning, recalibration, distillation, and quantization. The experiments show that it can, but the cost depends strongly on how deep the pruning goes and on the effort spent on recovery.

- **RQ1.** Yes. Removing 41.8% of the parameters keeps 101.1% of the teacher $AP_{50:95}$ (0.7199 against 0.7119) and 99.6% of AP_{50} (95.6% of the teacher fine-tuned with the same recipe), with a 1.4× CPU speed-up, 30% fewer GFLOPs, and a 1.7× smaller checkpoint, for one seed and 320 images.
- **RQ2.** The protected whole-backbone configuration keeps 101.1% of the AP of M_0 (95.6% of the fine-tuned teacher) at 41.8% reduction. At a matched 57.6% reduction with the final recipe, however, the uniform policy keeps 98.8%, the protected whole-backbone policy 85.4%, and the head-protected policy 82.9%. In our implementation, protecting the prediction modules did not improve compression at a given parameter count, and we did not see the uniform collapse reported for the reference.
- **RQ3.** We used Taylor scoring for all configurations. On ResNet-18 it was the best of four criteria (L1, L2, activation, Taylor) between 50% and 80% reduction and the weakest at 95%. We did not repeat the comparison on the detector, which is a limit of this work.
- **RQ4.** Distillation-based recovery brings back 44 to 59 AP points over the recalibrated students. The FPN feature term adds 1.7 AP points over fine-tuning alone on the aggressive student (one seed).
- **RQ5.** INT8 reduces the maximal model from 63.3 to 18.2 MB (9.1× smaller than M_0). Quantizing the stem is the costly step (20.6 retention points alone). With the stem in FP32 the INT8 student keeps 80.2% of the teacher AP (99.3% of its FP32 version), and it is 1.15× faster on the laptop in one session and 1.27 to 1.34× faster on a cloud CPU in three sessions. A repeat gave F1 = 0.811 for the INT8 model (0.815 in FP32).
- **RQ6.** The whole-backbone configuration is the preferred moderate point (41.8% parameter reduction, 101.1% retention). The aggressive configuration (57.6%, 85.4%) sits in between. The maximal configuration is the efficiency point (61.7% reduction, 45% fewer GFLOPs, 1.7× speed-up, 80.8% retention), and its INT8 version (18.2 MB, 80.2% retention) is the storage point.

Our central hypothesis was that where pruning is applied matters as much as how much is pruned. The first half holds: at the same reduction and recipe, the allocation changed retention by up to 15.9 points. The more specific idea that protecting the stem, RPN, and heads is the better allocation does not hold in our data, where uniform allocation was best. The recovery recipe mattered at least as much as the allocation.

7.2 Future Work

Closing the gaps of this study. Several results rest on one seed or one model. The allocation policies should be repeated at several reduction levels with several seeds, and the importance criteria, which we compared only on ResNet-18, should be run on the detector. The suspicion that the RoI head is the redundant part can be tested directly by pruning only the head to the same parameter budget.

Compression that knows what matters. The error analysis shows that the loss falls on the damage classes, not on the healthy ones. A recovery loss that weights the damage classes more, or a pruning search constrained by a minimum recall on those classes, would aim the compression at the quantity an inspection cares about instead of the average AP. The result that uniform allocation beat protection also suggests learning the per-group pruning ratios, for example by a multi-objective search over accuracy and measured latency, instead of fixing them by hand.

Beyond pruning a two-stage detector. Pruning a Faster R-CNN has a floor, because the RoI head and the region proposal network stay in the loop. A natural next step is cross-architecture distillation, with the large Faster R-CNN as teacher and a small one-stage or transformer-based detector as student, compared with

the pruned models at equal size and latency. On the quantization side, quantization-aware training and mixed precision could remove the stem problem we had to work around.

From a model to an inspection system. The compressed detector has to run on the drone, so the first measurement to make is the one of Appendix D: latency, memory, temperature, and energy per image on a Raspberry Pi 5 and a Jetson module, with the accelerator variants. Beyond that, a cascade that runs the small model on every frame and calls a larger one only on uncertain frames, or tracking across video frames to avoid detecting every frame, could bring the average cost down well below what any single model reaches. Both need data from real flights, and a check that the compressed model stays reliable under other weather, lighting, and sites, including how well its confidence scores are calibrated after compression.

Making the framework reusable. The framework prunes a detector and a classifier through the same interfaces. The next step is to publish it as a tested library, with adapters for other detectors and a continuous-integration test that checks the dependency graph on each new architecture, so that other groups can reproduce and extend these results.

Bibliography

[1] E. M. S. P. Veith, A. Nieße, and S. Lehnhoff, "AI in the Energy Sector: An Editorial Survey," *KI - Künstliche Intelligenz*, vol. 40, pp. 5–15, 2026. doi:10.1007/s13218-026-00903-5.

[2] M. S. S. Danish, "AI in Energy: Overcoming Unforeseen Obstacles," *AI*, vol. 4, pp. 406–425, 2023. doi:10.3390/ai4020022.

[3] E. Blasch, H. Li, Z. Ma, and Y. Weng, "The Powerful Use of AI in the Energy Sector: Intelligent Forecasting," 2021.

[4] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards Any Structural Pruning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[5] S. Das, J. Cheng, R. Ramnath, A. Rakshit, and J. Boubin, "EPIC: Efficient Pruning for Inference on Constrained Devices," in *PEARC*, 2025.

[6] M. Paganini and J. Forde, "Streamlining Tensor and Network Pruning in PyTorch," in *ICLR 2020 Workshop*, 2020. arXiv:2004.13770.

[7] N. Liu, X. Ma, Z. Xu, Y. Wang, J. Tang, and J. Ye, "AutoCompress: An Automatic DNN Structured Pruning Framework for Ultra-High Compression Rates," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[8] G. Shomron and U. Weiser, "Post-Training BatchNorm Recalibration," arXiv:2010.05625, 2020.

[9] R. Patra, R. Hebbalaguppe, T. Dash, G. Shroff, and L. Vig, "Calibrating Deep Neural Networks using Explicit Regularisation and Dynamic Data Pruning," arXiv:2212.10005, 2022.

[10] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge Distillation: A Survey," arXiv:2006.05525, 2021.

[11] B. Jacob, M. E. S. Tang, S. Kligys, M. Zhu, A. Howard, B. Chen, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," arXiv:1712.05877, 2017.

[12] S. Kim, G. Park, and Y. Yi, "Performance Evaluation of INT8 Quantized Inference on Mobile GPUs," *IEEE Access*, vol. 9, 2021.

[13] T. Wang, L. Yuan, X. Zhang, and J. Feng, "Distilling Object Detectors With Fine-Grained Feature Imitation," in *CVPR*, pp. 4933–4942, 2019.

[14] Y. Chen, P. Chen, S. Liu, L. Wang, and J. Jia, "Deep Structured Instance Graph for Distilling Object Detectors," in *ICCV*, pp. 4359–4368, 2021.

[15] Z. Zhao, X. Lv, Y. Xi, and S. Miao, "Defect Detection Method for Key Area Guided Transmission Line Components Based on Knowledge Distillation," *Frontiers in Energy Research*, vol. 11, 1287024, 2023. doi:10.3389/fenrg.2023.1287024.

[16] X. Huang, M. Jia, X. Tai, W. Wang, Q. Hu, D. Liu, P. Guo, S. Tian, D. Yan, and H. Han, "Federated Knowledge Distillation for Enhanced Insulator Defect Detection in Resource-Constrained Environments," *IET Computer Vision*, vol. 18, no. 8, pp. 1072–1086, 2024. doi:10.1049/cvi2.12290.

[17] H. Cui, D. Huang, W. Feng, Z. Li, Q. Ouyang, and C. Zhong, "FIAEPI-KD: A Novel Knowledge Distillation Approach for Precise Detection of Missing Insulators in Transmission Lines," *PLOS ONE*, vol. 20, no. 5, e0324524, 2025. doi:10.1371/journal.pone.0324524.

[18] S. Fan and B. Cao, "The Survey of Evolutionary Deep Learning-Based UAV Intelligent Power Inspection," *Drones*, vol. 10, no. 1, 55, 2026. doi:10.3390/drones10010055.

[19] S. Wei, Y. Cai, K. Dong, C. Liu, F. Yu, and S. Zhong, "Towards Autonomous Powerline Inspection: A Real-Time UAV-Edge Computing Framework for Early Identification of Fire-Related Hazards," *Drones*, vol. 10, no. 3, 183, 2026. doi:10.3390/drones10030183.

[20] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[21] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in *CVPR*, pp. 2117–2125, 2017.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *CVPR*, pp. 770–778, 2016.

[23] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.

[24] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, "Detectron2," 2019. https://github.com/facebookresearch/detectron2.

[25] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance Estimation for Neural Network Pruning," in *CVPR*, pp. 11264–11272, 2019.

[26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.

[27] Harmony Technology, internal reference report on Faster R-CNN compression (Prune–Recalibrate–Distill–Quantize), `rapporconf.pdf`. [Complete author, date, and title.]

Appendix A
Reproducibility Card

This appendix gives what is needed to rerun every number of Chapter 5: the commands, the settings of each stage, the validation gates, and the checkpoints. The scripts, the logs, and the SHA-256 of every checkpoint (file `CHECKPOINTS_SHA256.csv`) are in the project package.

A.1 Commands

All commands are executed from the project root; every stage writes a log and a JSON record next to its checkpoint.

```
# Teacher check and baseline evaluation
python verify_m0_8000.py

# M1: structured Taylor pruning (one run per configuration)
python run_m1_report.py prune --config expose_whole_backbone --target-reduction 41.8 --clip rank
python run_m1_report.py prune --config aggressive --target-reduction 57.6 --clip rank
python run_m1_report.py prune --config maximal --target-reduction 61.7 --clip rank

# M2: sequential recalibration
python run_m2_report.py recalibrate --m1 m1_report\checkpoints\M1R_<config>_t<target>_crank.pth

# M3: feature distillation
python run_m3_report.py distill --m2 m1_report\checkpoints\M2_<config>_t<target>_crank_matched.pth --iters 1600 --threads 4 --min
      -size 640,704,768,800 --max-size 1333 --freeze-at 0 --tag v5pure

# M4: INT8 post-training quantization
python run_m4_report.py quantize --m3 m1_report\checkpoints\M3_maximal_t61.7_crank_matched_v5pure.pth --threads 4 --fp32-units
      stem

# Controls: unpruned teacher fine-tuned with the same recipe, extra seeds, no feature term
python make_m0_control.py
python run_m3_report.py distill --m2 m1_report\checkpoints\M2_m0control.pth --iters 1600 --threads 4 --min-size 640,704,768,800
      --max-size 1333 --freeze-at 0 --tag v5pure
python run_m3_report.py distill --m2 <M2_aggressive> --iters 1600 --seed 1 --tag v5pure_seed1 --min-size 640,704,768,800 --max-
      size 1333 --freeze-at 0
python run_m3_report.py distill --m2 <M2_aggressive> --iters 1600 --beta 0 --tag v5pure_beta0 --min-size 640,704,768,800 --max-
      size 1333 --freeze-at 0

# Evaluation of any checkpoint, then costs, figures, tables, examples
python evaluate_m1_report.py --checkpoint <path-to-checkpoint.pth>
python measure_costs.py --threads 4 --repeats 20 --ckpt M0=default --ckpt M3_whole=<path> --ckpt M3_aggr=<path> --ckpt M3_max=<
      path>
python report_visuals.py figures
python report_visuals.py tables
python report_visuals.py examples --config <config> --n 4
```

A.2 Settings of each stage

Table A.1: Hyperparameters of each stage as used in the reported experiments.

Stage	Settings
M_1	Channel-level Taylor importance $\lvert W\,\partial L/\partial W\rvert$ accumulated without optimizer updates; scores clipped by rank (`-clip rank`), normalized per dependency group, and ranked globally; target reductions 41.8%, 57.6%, 61.7% of the parameters; stem, RPN, and RoI head protection according to the configuration.
M_2	Forward-only, sequential block-by-block recalibration of the frozen normalization layers on 300 calibration images (seed 1), followed by frozen rounds; acceptance window for the ratio of standard deviations student/teacher: [0.8, 1.25].
M_3	Teacher: frozen M_0. Loss $L_{\text{det}} + \beta\,\overline{\text{MSE}}_{P2\text{–}P5}$ with $\beta = 0.5$. SGD, momentum 0.9, weight decay 10^{-4}, learning rate 10^{-3}, 50 linear warm-up iterations then cosine decay, gradient clipping 10, batch size 2, seed 0, horizontal flip, final recipe: 1,600 iterations, training shortest edge in $\{640, 704, 768, 800\}$ (maximum 1333), all layers trainable (`freeze_at`=0), no class-logit term. Base recipe: shortest edge in $\{480, 512, 544, 576\}$ (maximum 960), stem and `res2` frozen (`freeze_at`=2), 800 iterations (1,200 for one aggressive run).
M_4	Static INT8 post-training quantization (PyTorch FX), 200 calibration images; stem, `res2`–`res5`, and RoI box head quantized per unit after folding FrozenBN into the convolutions; FPN, RPN, and predictor in FP32; backend `onednn`. Gates: exact folding (relative error $< 10^{-4}$), FPN cosine similarity > 0.9, finite outputs.
Evaluation	COCO protocol on the 320-image held-out subset; input shortest edge 800 px (maximum 1333); precision/recall/F1 at score threshold 0.5 and IoU 0.5.
Costs	fvcore GMACs (GFLOPs = 2×GMACs) for an 800 × 1200 input; latency: median of 20 runs after 3 warm-up runs, batch 1, 4 threads; size: FP32 `state_dict`.

A.3 Validation gates

Each stage ends with automatic gates whose outcome is stored in the JSON record of the checkpoint: shape and forward validity after pruning, convergence of the standard-deviation ratio after recalibration, unchanged parameter count and finite parameters after distillation, a decreasing training loss, exact folding and finite outputs before INT8 evaluation, and a checkpoint reload. Every checkpoint is evaluated on the identical 320-image held-out subset.

A.4 Checkpoints

Table A.2: Teacher and main compressed students (final recipe).

Model	Configuration	Parameters	Size (MB)
M_0 (8,000 it.)	teacher, SHA-256 `740FFEB4...B9BB8E`	41,305,311	165.6
M_3 whole backbone	41.8% reduction	24,026,537	96.4
M_3 aggressive	57.6% reduction	17,507,472	70.2
M_3 maximal	61.7% reduction	15,808,632	63.3
M_4 maximal	INT8, stem in FP32	15,808,632	18.2

Appendix B
Complete Results Ledger

Every evaluated checkpoint of the study on the 320-image held-out subset, with the recipe used. Retention is relative to M_0. Recipes: base = 800 iterations at 480–576 px with stem and `res2` frozen (1,200 iterations where stated); resolution, unfrozen, and class-logit = one change of the recipe on the base; combined = all changes with the class-logit term; final = the recipe used for the headline results (1,600 iterations). AP values are on the scale $[0, 1]$; the base rows of the aggressive and maximal students come from the original CPU runs.

Table B.1: All evaluated checkpoints.

Model	**Recipe**	**Red.**	$AP_{50:95}$	AP_{50}	AP_{75}	F1	**Ret.**
Teacher M_0	8,000 it.	0	0.7119	0.9215	0.8551	0.879	100.0%
M_0 fine-tuned (control)	final recipe	0	0.7528	0.9422	0.8824	0.896	105.7%
Residual-stream	final recipe	18.4%	0.7473	0.9412	0.8863	0.897	105.0%
Internal	final recipe	31.5%	0.7403	0.9330	0.8735	0.890	104.0%
Whole backbone	base	41.8%	0.6598	0.8695	0.7889	0.847	92.7%
Whole backbone	combined	41.8%	0.7176	0.9161	0.8508	0.879	100.8%
Whole backbone	final recipe	41.8%	0.7199	0.9183	0.8502	0.877	101.1%
Head protected	base	48.6%	0.6217	0.8323	0.7455	0.826	87.3%
Uniform	base	51.8%	0.6618	0.8669	0.7930	0.845	93.0%
Head protected	final recipe	57.6%	0.5901	0.7861	0.7025	0.808	82.9%
Uniform	final recipe	57.6%	0.7032	0.9001	0.8298	0.870	98.8%
Aggressive	base (800 it.)	57.6%	0.4877	0.6796	0.5894	0.765	68.5%
Aggressive	base (1,200 it.)	57.6%	0.5158	0.7109	0.6113	0.776	72.5%
Aggressive	base, GPU repeat	57.6%	0.4911	0.6858	0.5793	0.772	69.0%
Aggressive	Resolution	57.6%	0.5374	0.7291	0.6458	0.793	75.5%
Aggressive	Unfrozen layers	57.6%	0.5134	0.7121	0.6033	0.785	72.1%
Aggressive	Class-logit term	57.6%	0.5037	0.7000	0.6091	0.777	70.8%
Aggressive	combined	57.6%	0.6249	0.8194	0.7354	0.831	87.8%
Aggressive	final recipe, seed 0	57.6%	0.6079	0.8056	0.7248	0.823	85.4%
Aggressive	final recipe, seed 1	57.6%	0.6154	0.8158	0.7362	0.829	86.4%
Aggressive	final recipe, seed 2	57.6%	0.6070	0.8090	0.7153	0.827	85.3%
Aggressive	final recipe, $\beta = 0$	57.6%	0.5908	0.7900	0.6989	0.816	83.0%
Maximal	base	61.7%	0.4638	0.6519	0.5602	0.775	65.2%
Maximal	combined	61.7%	0.5843	0.7771	0.6907	0.817	82.1%
Maximal	final recipe	61.7%	0.5751	0.7692	0.6785	0.816	80.8%
Maximal, INT8 stem only	final recipe + PTQ	61.7%	0.4289	0.6146	0.5071	–	60.2%
Maximal, INT8 final	final recipe + PTQ	61.7%	0.5712	0.7650	0.6732	–	80.2%
Maximal, second training	final recipe	61.7%	0.5770	0.7703	0.6809	0.815	81.1%
Maximal, second training, INT8	final recipe + PTQ	61.7%	0.5738	0.7651	0.6843	0.811	80.6%

Appendix C
Additional Baseline Analyses of the Teacher

This appendix gathers complementary analyses of the baseline M_0 that support the error analysis of Section 5.8.

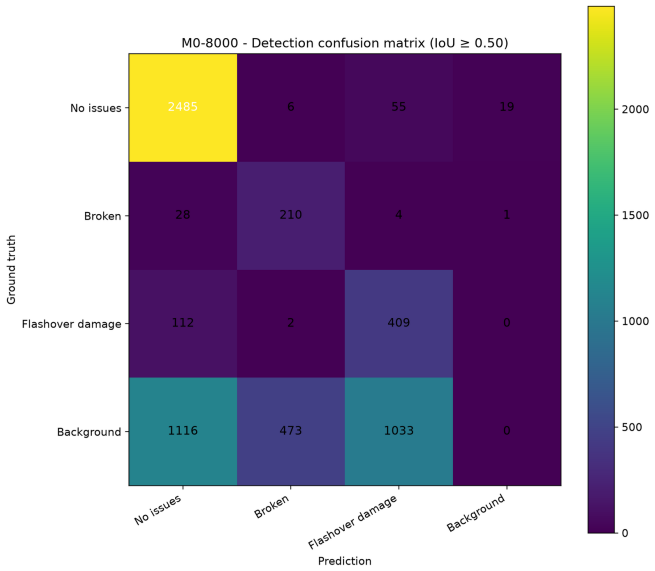


Figure C.1: Three-class confusion matrix of M_0 on the held-out subset.

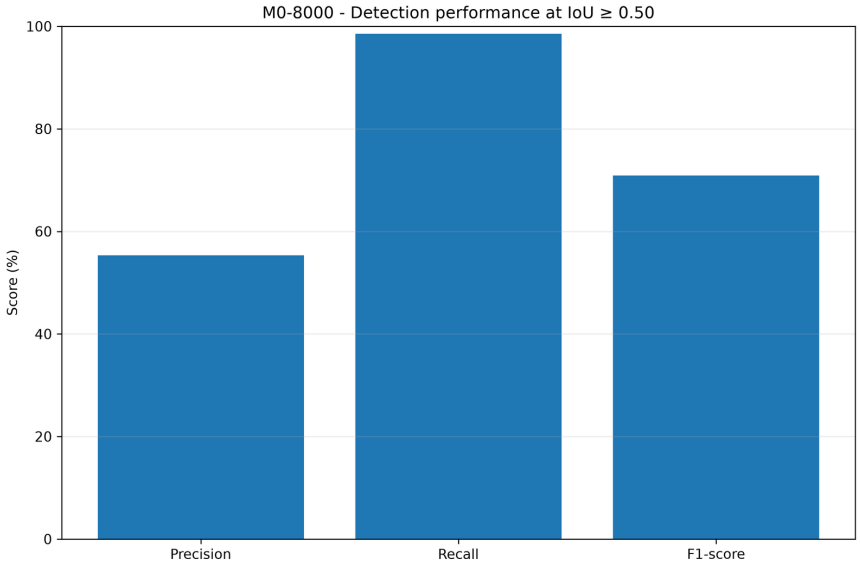


Figure C.2: Precision, recall, and F1 of M_0 per class.

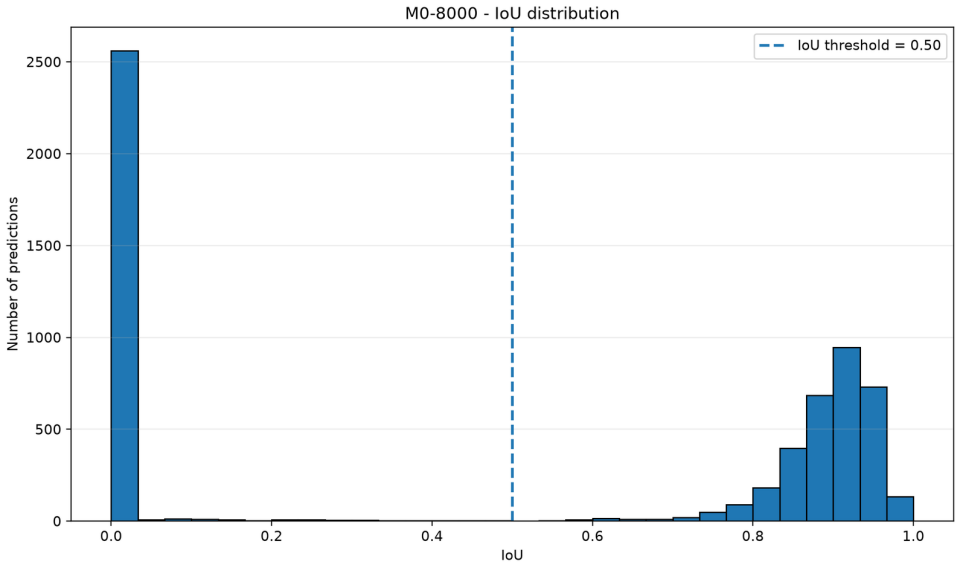


Figure C.3: Distribution of the IoU between matched predictions and ground-truth boxes for M_0.

Appendix D
Embedded Platforms and Integration Path

All latencies in this work come from a laptop CPU and a cloud CPU. This appendix describes the embedded platforms the compressed detector is meant for, looks at the Raspberry Pi in more detail, and lists the steps that would take the framework from the workstation to a model running on the board. We have not run anything on an embedded board yet, so what follows is a plan. Section D.4 lists what would have to be measured.

D.1 What an embedded platform is

An embedded platform is a small computer that the drone can carry and power from its battery. Its memory is fixed, its power budget is limited, and active cooling is not guaranteed. Four classes are relevant for a detector of this size (Table D.1).

Table D.1: Classes of embedded platforms and their fit with a Faster R-CNN detector. Hardware figures are those given by the manufacturers for the examples; the last column is our assessment, not a measurement.

Class	Example	Typical power	Fit with our detector
Microcontroller	Cortex-M boards with some hundreds of kilobytes of memory	below 1 W	Not possible: the smallest model is 18.2 MB
Single-board computer, CPU only	Raspberry Pi 5: quad-core Arm Cortex-A76 at 2.4 GHz, 8 GB RAM in the reference kit	a few watts, 45 W supply for the full kit	Possible in memory; speed to be measured
Single-board computer with accelerator	Raspberry Pi 5 with the AI HAT+ (Hailo accelerator, 13 or 26 TOPS, PCIe)	a few watts more	Possible, needs a model conversion for the accelerator
GPU module	NVIDIA Jetson Orin Nano: 20 TOPS (4 GB) or 40 TOPS (8 GB), 7 to 15 W	7 to 15 W	Most direct route for a PyTorch detector; heavier and more power

D.2 The Raspberry Pi

The Raspberry Pi is a credit-card-sized computer that runs Linux on an Arm processor. The Raspberry Pi 5 has a quad-core Arm Cortex-A76 at 2.4 GHz and comes with several memory sizes (the reference kit has 8 GB), a microSD slot, USB, a camera connector, and a PCIe connector for an optional accelerator board. It has no CUDA GPU, so the detector would run on the CPU. This is why our CPU measurements are the relevant ones, and why quantization matters: INT8 arithmetic reduces both the memory and the work per operation on an Arm CPU. Figure D.1 shows the onboard unit we have in mind. The supplier figures come from public product pages[1].

[1] Raspberry Pi kit page of a reseller: `canakit.com`, Raspberry Pi 5 AI kit; NVIDIA Jetson Orin Nano datasheet.

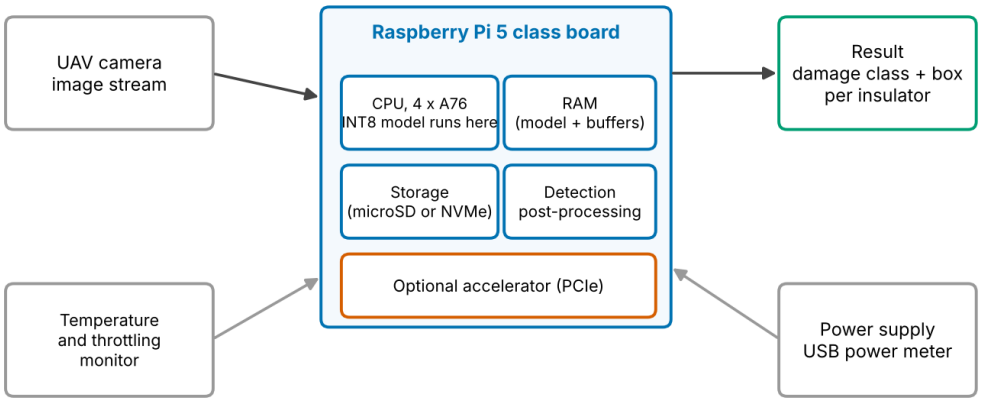


Figure D.1: Onboard unit built around a Raspberry Pi 5 class board: the camera stream enters the board, the INT8 student runs on the CPU (or on an optional accelerator), and the result is a class and a box for each insulator. Temperature, throttling, and power are monitored from outside.

Memory. Our models are small next to the 8 GB of the reference kit: the FP32 detector is 165.6 MB, the maximal student 63.3 MB, and its INT8 version 18.2 MB (Figure D.2). Memory is not the constraint. The working memory of the network, the feature maps of a large image, is bigger than the weights and still has to be measured. Speed is the open question. On our laptop CPU the maximal INT8 model needs 1.6 to 4.5 seconds per image depending on the machine (Section 5.6), and a result on one processor does not transfer to the Raspberry Pi.

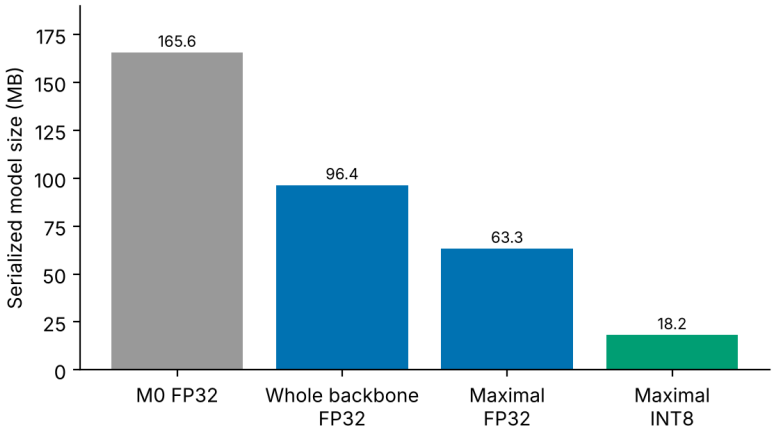


Figure D.2: Serialized size of the detector and of the compressed students (measured values).

D.3 How the framework would be integrated

The steps follow Figure D.3. The framework already does steps 1 to 4; the others are additions.

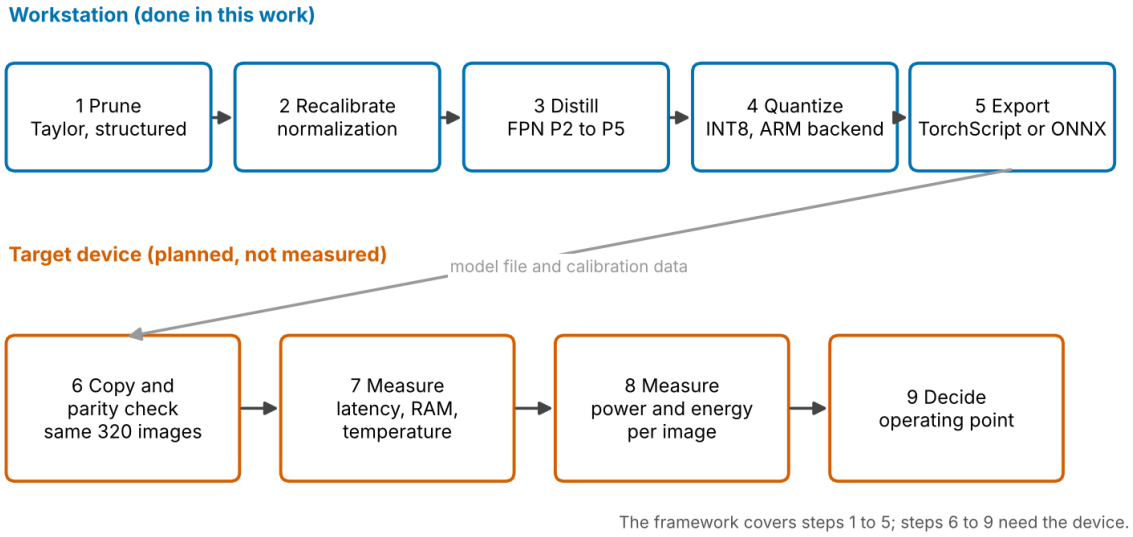


Figure D.3: From the framework to a measured model on the device. The blue steps exist in this work; the orange steps are planned.

1. Fix the budget. Choose the target before compressing: time per image, memory limit, and power. They decide which operating point of Table 5.10 is acceptable.
2. Compress on the workstation. Prune, recalibrate, and distill with the framework as in this work. Nothing changes, since the framework works on the model and not on the device.
3. Quantize for the Arm backend. We ran INT8 with the x86 backends (`onednn` on the laptop, `fbgemm` on the cloud CPU). An Arm processor uses `qnnpack`. The quantization script already takes a backend option and falls back to `qnnpack` when no x86 backend is available, so we would rerun it with that backend and the same 200 calibration images, and check the accuracy on the workstation before moving the model.
4. Keep the stem in floating point, since the sensitivity measurement of Section 5.6 also applies on the device.
5. Export. The pruned and quantized network has to be converted to a format that runs without Detectron2, either TorchScript or ONNX. This is the main technical risk. Faster R-CNN has operations whose size depends on the number of detections (proposal selection and non-maximum suppression), and we have not tried exporting the Detectron2 model. If it fails, we would export the backbone and the heads and keep the post-processing in plain code.
6. Run on the device. Install a PyTorch or ONNX Runtime build for 64-bit Arm, copy the model, and run with four threads as in our laptop measurements.
7. Parity check. Evaluate the same 320 test images on the device and compare the AP with the workstation value. A difference larger than the run-to-run variation (0.2 AP points, Section 5.6) would mean that the export or the Arm quantization changed the model.
8. Measure, using the protocol below.
9. Optional accelerator. A Hailo accelerator has its own compiler, which starts from an ONNX model and does its own quantization. The framework would provide the pruned ONNX model. We do not know whether the whole detector, region proposal and box head included, can run on the accelerator; the likely arrangement is the backbone on the accelerator and the rest on the CPU.

In the code this would be a small deployment module next to the evaluation tools: an export function, the device benchmark script delivered with this work (`benchmark_on_device.py`), and a command for the parity check. We have only tested the benchmark script on a workstation.

D.4 Measurement protocol on the device

Before claiming embedded deployment, these quantities have to be measured. The benchmark script records the first four; power needs an external meter.

- Latency: median and 95th percentile over at least 30 runs after a warm-up, with real images, because

post-processing time depends on the number of candidate boxes.
- Peak memory: the resident memory of the process, which includes the runtime libraries and is therefore larger than the model file.
- Temperature and throttling: the processor temperature before and after a long run, and the throttling flag of the board, because a processor that slows itself down gives a latency that cannot be sustained.
- Accuracy: AP and F1 on the 320 images, for the parity check.
- Power and energy: the average power read on a USB power meter during inference, and the energy per image, obtained by multiplying it by the median latency.

The result would turn the statement in the Discussion chapter, that a few seconds per image on a laptop CPU is not real time, into a statement about the target device.

Appendix E
Register of Deviations and Open Items

Table E.1: Declared differences with the reference experimental protocol.

Aspect	Reference report	This study
Framework	torchvision Faster R-CNN	Detectron2 Faster R-CNN R50-FPN
Baseline budget	5,000 iterations (AP 0.8314)	8,000 iterations (AP 0.7119)
Pruning granularity	as described in the reference	structured channel pruning with Taylor importance and rank-normalized global ranking
Dependency handling	—	in-house dependency graph (DepGraph principle)
Distillation budget	not specified	1,600 iterations at 640–800 px, $\beta = 0.5$ (800 and 1,200 iterations in the base runs)
INT8 backend	—	PyTorch FX, `onednn`, partial INT8
Zero-out mechanism	filters zeroed then dropped	tested, not reproduced; channel ranking with rank clip kept
Uniform policy	fc6 and fc7 collapse (-99 and -98%), 61.7% retention	rank-normalized equal weights, about 45% of the channels of each group removed, fc layers not emptied; 98.8% retention at 57.6%
Class-logit KD	not part of the loss	combined only, reported as an extension; headline numbers use final recipe
INT8 coverage	stem, `res2`–`res5`, box head	stem kept in FP32 (quantized alone it costs 20.6 retention points); the final model retains 80.2%
ResNet-18 study	four criteria on a COCO subset, KD against CE, re-initialization	Imagenette, four criteria, KD against CE and re-initialisation run through the framework pruner; Torch-Pruning direct only as a control
Comparison basis	AP	retention and relative reductions only

Table E.2: Declared differences for the ResNet-18 study (Section 5.1).

Aspect	This study
Data and input	Imagenette (10 classes), 128×128 inputs, 8,000 training and 2,500 validation images
Baseline	ImageNet-pretrained ResNet-18, 8 epochs of fine-tuning
Pruning	framework pruner (Taylor, global ranking, protected classifier), 81% target; direct Torch-Pruning as control
Recovery	distillation from the unpruned FP32 network (identical architecture), 15 epochs (4 also reported)
INT8	PyTorch FX static PTQ, 500 calibration images
Latency	indicative only (single run, shared GPU machine)
Replicates	one seed, 2,500 validation images

Still open: the importance-criteria comparison on the detector, several seeds for the allocation policies, a measured study of the causes of the failure cases (the images were reviewed visually, Section 5.8.1), and an energy, RAM, and temperature study on embedded hardware.